

MODUL

PEMROGRAMAN BERBASIS OBJECT
DENGAN JAVA NETBEANS



Evita Fitri, M.Kom
0301029701

FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS NUSA MANDIRI
2023



UNIVERSITAS NUSA MANDIRI

Gedung Rektorat : Nusa Mandiri Tower Jl. Jatiwaringin Raya No. 2, Jakarta Timur 13620
Telp. (021) 28534471, 28534390 e-mail : rektorat@nusamandiri.ac.id

SURAT TUGAS

No. 006/3.01/UNM/WR/III/2023

Wakil Rektor I Bidang Akademik Universitas Nusa Mandiri menugaskan kepada:

Nama : Evita Fitri, M.Kom
NIP : 202201006
NIDN : 0301029701

Untuk pengerjaan Penyusunan Modul Ajar Semester Genap 2022-2023

Judul Modul Ajar : Pemrograman Berbasis Object Dengan Java NetBeans
Masa Penugasan : Maret 2023 s/d April 2023

Demikianlah penugasan ini agar dapat dilaksanakan sebagaimana mestinya. Atas perhatian dan kerjasamanya kami mengucapkan terima kasih.



Jakarta, 03 Maret 2023
Wakil Rektor I Bidang Akademik

Nita Merlina, M.Kom

Tembusan :

1. Divisi SDM
2. Wakil Rektor I Bidang Akademik
3. Wakil Rektor II Bidang Non Akademik
4. Ka. Prodi
5. Ybs



UNIVERSITAS NUSA MANDIRI

• Jl. Kramat Raya No. 18, Jakarta Pusat
• Nusa Mandiri Tower,
Jl. Margonda Raya No. 545, Depok

• Jl. Damai No. 8, Warung Jati Barat (Margasatwa), Jakarta Selatan
• Jl. Daan Mogot No. 31, Tangerang

DAFTAR ISI

COVER	Error! Bookmark not defined.
DAFTAR ISI.....	ii
BAB I LANDASAN TEORI DASAR PEMROGRAMAN BAHASA JAVA	Error! Bookmark not defined.
1.1. Pemrograman Java	Error! Bookmark not defined.
1.2. Sifat dan Karakteristik Pemrograman Java.....	Error! Bookmark not defined.
1.3. Konsep-Konsep Pemrograman Berbasis Object.....	3
1.4. Karakteristik Pemrograman Berbasis Object.....	5
1.5. Jenis-Jenis Pemrograman Java.....	8
1.6. Pengenalan Program Java Netbeans	8
BAB II PERINTAH DASAR PADA PEMROGRAMAN JAVA	12
2.1. Pengenalan Tipe Data	1Error! Bookmark not defined.
2.2. Penerapan Perintah Tipe Data Dasar Pada Java Netbeans.....	15
BAB III TEORI & KONSEP PEMROGRAMAN BEBRBASIS OBJECT	2Error! Bookmark not defined.
3.1. Penerapan Karakteristik Berorientasi Object Pada Java.....	24
BAB IV OPERATOR PADA BAHASA JAVA.....	32
4.1. Operator Aritmatika Pada Java	32
4.2. Operator Pemberi Nilai	33
4.3. Operator Penambah dan Pengurang.....	34
4.4. Operator Pembandingan	35
4.5. Operator Logika	36
BAB V PERINTAH MASUKAN, PENYELEKSI KONDISI, PERULANGAN DAN ARRAY PADA PEMROGRAMAN JAVA	38
5.1. Perintah Masukan Pada Program Java.....	38
5.2. Perintah Seleksi Kondisi	41
5.3. Perintah Perulangan Pada Pemrograman Java.....	43
5.4. Perintah Array Pada Pemrograman Java.....	47
BAB VI CONTOH LATIHAN STUDI KASUS	53
DAFTAR PUSTAKA.....	60

BAB I

LANDASAN TEORI DASAR PEMROGRAMAN BAHASA JAVA

1.1. Pemrograman Java

Java merupakan bahasa pemrograman yang sangat populer dikarenakan aplikasi yang bisa dibuat menggunakan bahasa ini sangatlah luas, mulai dari aplikasi di komputer maupun smartphone (Enterprise 2016). James Gosling, Patrick Naughton, Chris Warth, Ed Frank, dan Mike Sheridan dari Sun Microsystems, Inc. menemukan bahasa pemrograman Java, yang pertama kali dikenal sebagai bahasa pemrograman OAK, pada tahun 1991 ketika perusahaan meluncurkan Green Project (sebuah proyek penelitian untuk mengembangkan bahasa pemrograman yang dapat dijalankan di berbagai platform). Sebelumnya, aplikasi yang dibuat untuk sistem operasi (dan perangkat keras) tertentu hanya dapat berfungsi dengan baik pada sistem operasi tersebut, sehingga hal ini merupakan kemajuan yang signifikan. Dengan kata lain, sangat tidak mungkin untuk mengeksekusi perangkat lunak yang dibuat untuk satu sistem operasi, seperti Windows, di sistem operasi lain, seperti Unix/Linux. James Gosling dari Sun Microsystems meraih kesuksesan besar dengan usahanya. Saat ini, Java mungkin sudah terbiasa. Asapun jenis-jenis dari java yang telah dikembangkan sebagai berikut:

- java.lang: Peruntukan kelas elemen-elemen dasar.
- java.io: Peruntukan kelas input dan output, termasuk penggunaan berkas.
- java.util: Peruntukan kelas pelengkap seperti kelas struktur data dan kelas kelas penanggalan.
- java.net: Peruntukan kelas TCP/IP, yang memungkinkan berkomunikasi dengan komputer lain menggunakan jaringan TCP/IP.
- java.awt: Kelas dasar untuk aplikasi antarmuka dengan pengguna (GUI)
- java.applet: Kelas dasar aplikasi antar muka untuk diterapkan pada penjelajah web.

Pada Java, terdapat beberapa tipe karakteristik Pemrograman Java, diantaranya berbasis object, terdistribusi, multi platform dan multithread.

1.2. Sifat dan Karakteristik Pemrograman Java

Seperti yang dijelaskan sebelumnya, pada Java, terdapat beberapa karakteristik Pemrograman Java, diantaranya :

a. Dinamis

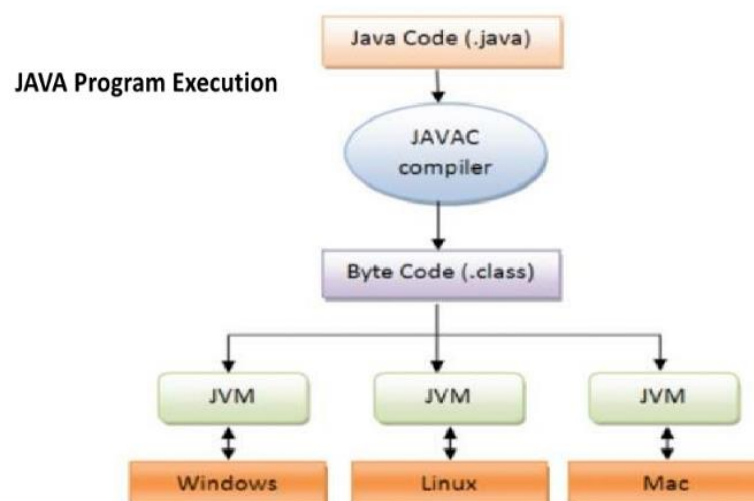
Java dibuat dengan mempertimbangkan fleksibilitas untuk lingkungan pengembangan. Hal ini menyiratkan bahwa menambahkan properti atau metode ke sebuah kelas cukup sederhana dan tidak akan mempengaruhi program yang menggunakan kelas tersebut.

b. Berorientasi Object

Pemrograman berorientasi objek adalah gagasan mengembangkan program dengan mengelompokkan masalah ke dalam objek-objek berbeda, sederhananya. Setiap objek tampak ada di dunianya sendiri, yang dapat memuat atribut dan metode (istilah lain untuk variabel dan operasi internal objek). Bersama-sama, berbagai komponen ini membentuk satu program. Pemrograman berorientasi objek sering disebut dengan singkatan bahasa Inggrisnya, OOP (Object Oriented Programming), atau dengan singkatan PBO (Programming Based on Objects). Proses penulisan program dengan menggunakan ide "objek" yang berisi data dan kode dikenal sebagai pemrograman berorientasi objek (OOP). Informasi ini tersedia dalam bentuk field (juga dikenal sebagai atribut atau properti dalam konteks yang lebih informal) dan kode dalam bentuk prosedur (juga dikenal sebagai metode).

c. Architectural Neutral

Java is architecture-neutral



Program Java tidak bergantung pada platform karena kompiler java menghasilkan format file objek yang tidak bergantung pada arsitektur yang berbeda, sehingga dapat dijalankan di berbagai platform yang sudah memiliki JVM hanya dengan satu versi.

d. Multithreaded.

Java memiliki fitur interaktif yang memungkinkan untuk membuat program yang dapat melakukan beberapa aktivitas sekaligus tanpa harus bertempur dengan langkah berikutnya.

e. Performa Tinggi

Program Java menggunakan kompiler secara langsung, yang berkontribusi pada kinerja tinggi. Namun, ada berbagai kompiler Java dari Inprise, Microsoft, atau Symantec yang dapat Anda gunakan untuk meningkatkan kinerja Java.

f. Platform Independen

Sebagai bytecode yang tidak bergantung pada platform, Java dapat dijalankan di berbagai sistem operasi, termasuk Microsoft Windows, Linux, BSD, Macintosh, dan lain-lain. Sebagai hasilnya, Java tidak bergantung pada satu platform saja.

g. Terinterpretasi

Agar dapat diterjemahkan oleh sistem apa pun yang berisi program Java, kode bit Java segera diterjemahkan ke perintah mesin. JVM dan Java Api membentuk Java sebagai sebuah platform.

h. Terdistribusi

Sebuah aplikasi Java dapat dikirimkan dengan cepat berkat pustaka jaringan karena bahasa Java dibuat untuk konteks distribusi internet.

i. Portabel

Sebuah aplikasi Java dapat dikirimkan dengan cepat berkat pustaka jaringan karena bahasa Java dibuat untuk konteks distribusi internet.

j. Sturdy And Secure

kuat dan amanPenanganan eksepsi runtime dalam pemrograman Java memungkinkan untuk menghilangkan kesalahan dengan melakukan pengecekan pada saat kompilasi dan selama eksekusi. Java dibuat agar mudah digunakan dan mudah dipelajari. Tata bahasa Java sebanding dengan C++ tetapi telah ditingkatkan secara substansial, terutama dalam pewarisan berganda.

Java memiliki banyak fitur keamanan yang memungkinkan Anda untuk mengembangkan sistem dan aplikasi yang kuat dan bebas dari virus.

1.3. Konsep-Konsep Pemrograman Berbasis Object

Konsep pemrograman berbasis Object terdapat beberapa, diantaranya ialah mengenai Method, Clas dan juga Objek.

a. Object dan Class

Pemrograman berorientasi objek mendasarkan persepsinya tentang dunia pada objek. Dalam kehidupan sehari-hari, objek berfungsi sebagai cerminan pola kerja manusia. Ada dua (2) hal yang dapat dilihat dalam sebuah objek, yaitu:

Attribut : Apa pun yang terkait dengan Objek disebut sebagai atribut. Atribut adalah Variabel atau Anggota dalam aplikasi atau perangkat lunak. Objek Burung, sebagai ilustrasi. Paruh, ekor, sayap, kaki, mata, dan fitur-fitur lainnya merupakan atribut dari burung tersebut.

Behaviour : Perilaku adalah pola tingkah laku atau tingkah laku yang dimiliki oleh suatu objek. Sebagai contoh, objek Burung dapat berjalan, mengepakkan sayap, dan terbang, dan lain-lain. Behavior adalah sebuah Method atau Fungsi dalam aplikasi program. Bentuk penulisan class, sebagai berikut :

```
[public | private] [abstract] Class Nama_Class
{
... daftar property...
... daftar Method ...
}
```

Bentuk penulisan mendeklarasikan Object, dengan menggunakan new, seperti dibawah ini :

```
nama_Class nama_objek = new nama_Class();
```

nama_Class, merupakan nama Class yang akan dijadikan objek.

nama_objek, merupakan nama objek baru.

Contoh pembuatan Class sederhana :

```
Class burung
```

```
{
String jenis, warna;
```

```
4
```

```
int usia;
```

```
}
```

```
Class burung_terbang
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
//membuat objek
burung burung_elang = new burung();

.....

.....

}
```

b. Method

Metode adalah kumpulan pernyataan di dalam kelas yang menangani tugas tertentu. Menggunakan metode adalah cara objek berkomunikasi satu sama lain. Method adalah implementasi operasi yang bisa dilakukan oleh Class dan Object. Operasi operasi yang dilakukan oleh Methode, diantaranya, yaitu :

1. Sebuah Metode dapat menerima dan memodifikasi data atau bidang di dalam Metode.
2. Nilai sebuah objek dapat dipengaruhi oleh sebuah metode.

Berikut bentuk penulisan deklarasi Method:

```
Tipe_Akses Tipe_Return NamaMethod(Argumen1, Argumen2,...,Argumen-N)
{
... Badan / Tubuh Method ..
}
```

Berikut penjelasan deklarasi Methode diatas :

- ✓ Tipe Akses, menyatakan tingkatan akses untuk memproteksi akses terhadap data-data didalam Method, tipe akses ini bersifat opsional.
- ✓ Tipe Return, menyatakan nilai hasil yang diolah oleh Method akan dikembalikan atau akan mengirimkan kepada objek yang memanggil Method. Bentuk Tipe Return, bisa berupa tipe data primitive yaitu integer, float, double dan lain-lain.

1.4. Karakteristik Pemrograman Berbasis Objek.

Berikut karakteristik-karakteristik pemrograman berbasis object diantaranya ;

a. Enkapsulasi (Encapsulation)

Enkapsulasi atau pembungkus, adalah mekanisme untuk menghubungkan metode dan properti yang nantinya akan oleh dan melindungi dari manipulasi eksternal dan penyalahgunaan oleh pengguna kelas. Dalam enkapsulasi, pemrograman objek memfasilitasi pelacakan bug. Konsep enkapsulasi meminimalkan kesalahan

pengkodean, yang juga berkontribusi pada modularitas, memungkinkan satu kelas untuk berintegrasi dengan kelas lainnya (Hartono 2023). Enkapsulasi merupakan proses membundel data dengan metode adalah proses yang dikenal sebagai enkapsulasi, dan ini efektif untuk mengaburkan detail implementasi dari pengguna. Enkapsulasi berfungsi untuk mencegah akses sewenang-wenang atau intervensi dalam proses program oleh program lain. Ide enkapsulasi sangat penting untuk mempertahankan program sambil menjaga agar tuntutan program tetap dapat diakses setiap saat. Kita ambil contoh, pada saat anda mengganti chanel TV menggunakan remote TV, apakah anda mengetahui proses yang terjadi didalam TV tersebut ?, maka jawabannya tidak tau, dan anda pun sebagai pembeli TV tidak mau dipusingkan dengan proses yang terjadi. Maka hal tersebut menyederhakan sistem.

Contoh dalam program :

Belajar.Java

```
class belajar{  
    public String x ="Pintar";  
    private String y = "Java";  
}
```

Pintar.Java

```
public class Pintar{  
    public static void main(String[]args){  
        Coba panggil = new Belajar();  
        System.out.println("Panggil X : "+panggil.x);  
        System.out.println("Panggil Y : "+panggil.y);  
    }}
```

b. Pewarisan (Inheritance)

Ide pewarisan dalam pemrograman memungkinkan untuk 'mewariskan' properti dan metode sebuah kelas ke kelas lain. Memanfaatkan kemampuan "penggunaan ulang kode" untuk mencegah duplikasi kode komputer membutuhkan ide pewarisan.

Dalam kode perangkat lunak, ide pewarisan mengatur kelas-kelas dalam sebuah hirarki. Kelas yang akan "diwarisi" dapat disebut sebagai kelas dasar, kelas induk, atau kelas super. Kelas yang 'menerima warisan' dapat disebut sebagai kelas anak, subkelas, kelas turunan, atau kelas pewaris. Sebagai contoh, pada saat kita bicara

mengenai bus, maka bus tersebut bisa mewariskan kepada bus yang lain berupa, nomor trayek, body besar, jumlah penumpang banyak dan lain sebagainya.

c. Polymorphism

Sebuah kelas dapat 'mewariskan' atribut dan fungsinya kepada kelas lain dengan menggunakan konsep pemrograman pewarisan. Untuk mencegah duplikasi kode program, ide pewarisan digunakan untuk memanfaatkan fitur "penggunaan ulang kode".

Dalam kode perangkat lunak, sebuah hirarki atau struktur kelas dibuat dengan menggunakan ide pewarisan. Kelas induk, kelas super, atau kelas dasar dapat digunakan untuk mendeskripsikan kelas yang akan "diwarisi". Sedangkan kelas yang 'menerima warisan' dapat disebut sebagai kelas anak, subkelas, kelas turunan, atau kelas pewaris.

Pada polymorphism kita akan sering menjumpai 2 (dua) istilah yang sering digunakan dalam pemrograman berbasis objek, istilah tersebut yaitu :

a) Overloading.

Overloading yaitu menggunakan 1 (satu) nama objek untuk beberapa Method yang berbeda ataupun bisa juga beda parameternya.

b) Overriding

Overriding akan terjadi apabila ketika pendeklarasian suatu Method subClass dengan nama objek dan parameter yang sama dengan Method dari superClassnya.

d. Abstrak

Abstrak, yang dimaksudkan untuk melihat sebuah sistem, menjadi lebih sederhana atau lebih sederhana dalam pemrograman berbasis objek. Jika kita memeriksa sebuah sistem, seperti sepeda motor, kita dapat melihat bahwa sepeda motor pasti memiliki sistem pengapian, sistem rem, sistem persneling, dan sistem lainnya. Ketika kita menggabungkan semua sistem ini menjadi satu, yaitu sistem sepeda motor, maka akan menjadi lebih mudah.

e. Modularity

Kemampuan untuk menulis atau membuat sebuah objek secara independen dari objek lain merupakan fitur pemrograman berbasis objek. Agar lebih mudah dalam

mendesain dan memodifikasi aplikasi. Mari kita gunakan sistem sepeda motor sebagai contoh. Jika sistem rem terintegrasi langsung ke dalam komponen utama sepeda motor, setiap perbaikan atau modifikasi akan membutuhkan pembongkaran komponen utama sebelum mencapai komponen sekunder, yang akan memakan banyak waktu. Untuk memperbaiki atau memodifikasi sebuah objek, cukup lakukan perjalanan ke objek yang dituju berkat modularitas.

1.5. Jenis-Jenis Pemrograman Java

Dua kategori program Java adalah applet dan aplikasi:

a. Applet

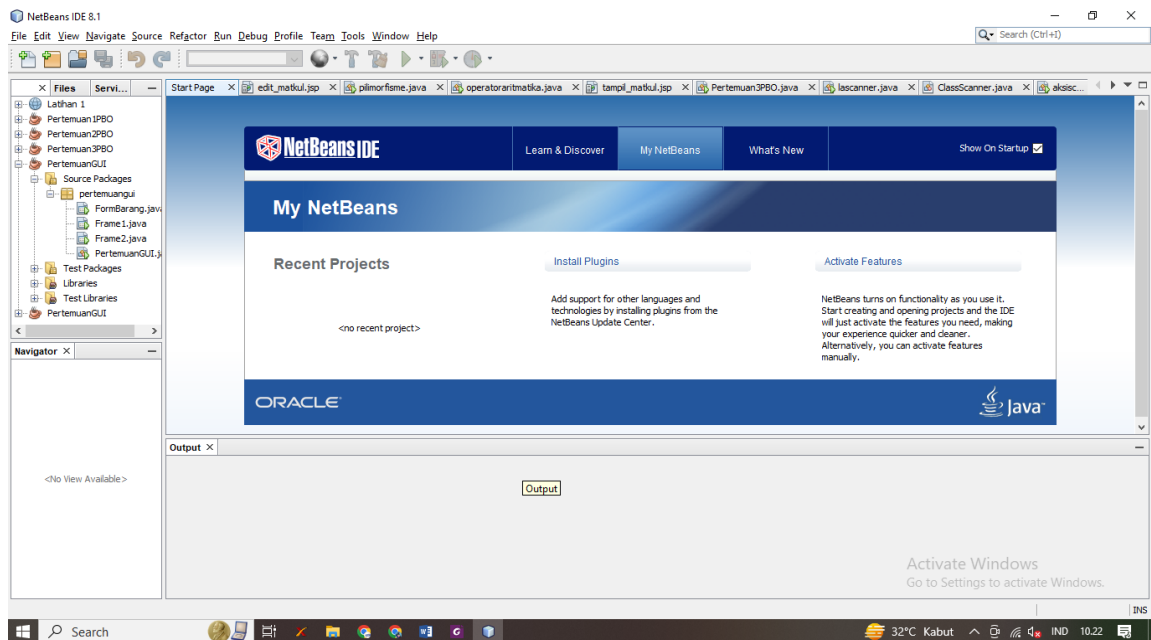
Applet adalah program Java yang dapat diunggah ke server web dan dijelajahi menggunakan browser web. Dalam hal ini, browser yang mampu menjalankan Java digunakan (seperti Netscape Navigator, Internet Explorer, atau Mozilla Firefox).

b. Implementasi

Aplikasi adalah program serba guna yang dibuat dengan Java. Tidak ada persyaratan untuk perangkat lunak browser untuk meluncurkan aplikasi; mereka dapat dijalankan secara langsung. Aplikasi dapat dibandingkan dengan program yang ditulis dalam bahasa C atau Pascal. Anda dapat langsung menjalankannya setelah kompilasi.

1.6. Pengenalan Program Java NetBeans

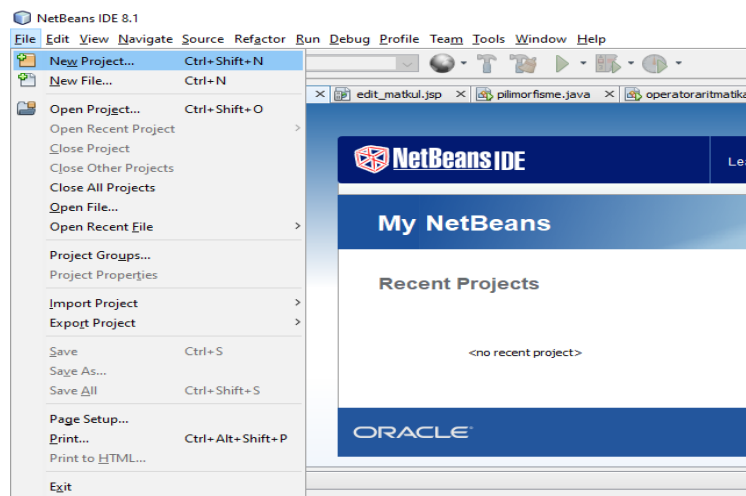
Pada sesi ini, kita akan membangun program di bidang ini dan mempelajari tentang program Java yang tersedia. Seperti disebutkan secara singkat di atas, ada dua (dua) kategori utama program Java: Aplikasi Java dan Applet Java. Dalam penjelasan berikut, lebih banyak lagi yang akan dibahas dan juga diterapkan pada editor Java. Untuk saat ini penulis memanfaatkan download gratis Netbeans editor versi 8.1. File Program Java adalah File Program dengan ekstensi .java yang dapat dikompilasi, dijalankan, dan digunakan untuk menampilkan hasil.



Gambar 1. Tampilan Awal NetBeans 8.1

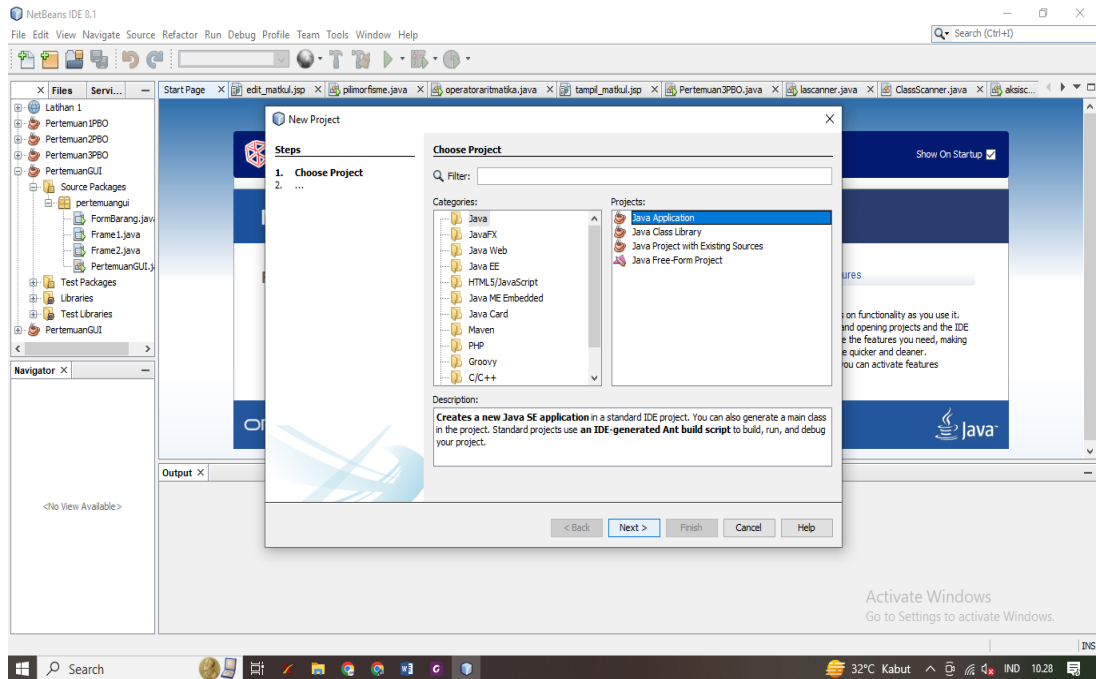
1. Pembuatan File Baru Pada Java NetBeans

Langkah awal pada pembuatan file atau project baru klik New pada bagian File – New Project.



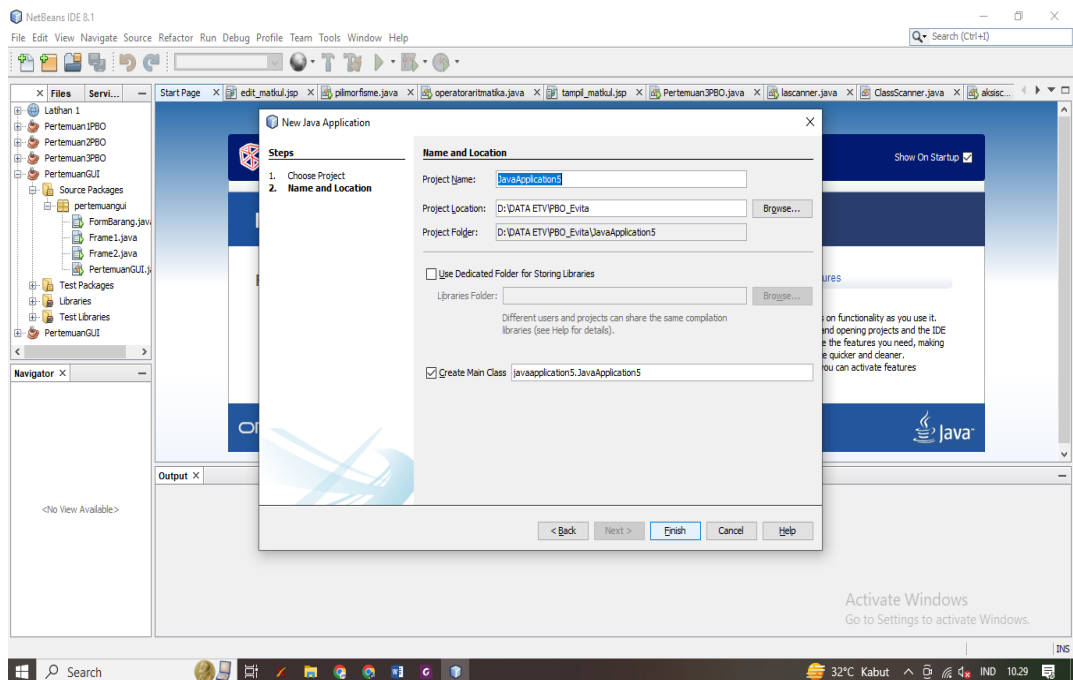
Gambar 2. Pembuatan Project Baru

Selanjutnya akan Tampil Jendela New Project, pilih Java Pada Categories dan Java Application Pada Project, kemudian klik next.



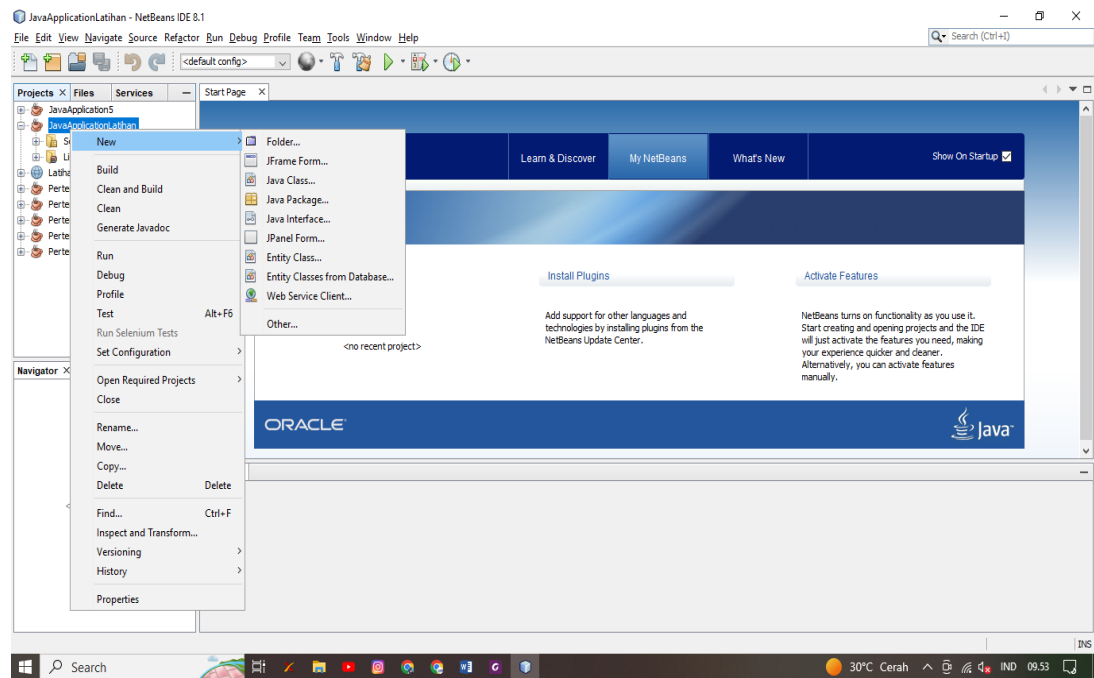
Gambar 3. Pemilihan type Project Baru

Setelah itu akan tampil Jendela New Java Application, tentukan nama Project dan juga Folder penyimpanannya, kemudian klik Finish.



Gambar 4. Detail Nama Project dan Penyimpanan Project

Selanjutnya akan tampil detail project baru yang kita buat dan terdapat jendela project, pada tampilan tersebut, kita dapat membuat class, package, jpanel, jframe dan juga lainnya sesuai kebutuhan dalam membuat project.



Gambar 5. Detail Pembuatan Type File-File Project Java

BAB II

PERINTAH DASAR PADA PEMROGRAMAN JAVA

2.1. Pengenalan Tipe Data

Delapan tipe data primitif tersedia di Java, termasuk empat untuk bilangan bulat, dua untuk bilangan pecahan, dan dua yang terakhir untuk karakter dan Boolean. Adapun tipe data dapat dilihat pada gambar tabel berikut :

Tipe Data	Ukuran Memori	Jangkauan Nilai	Jumlah Digit
Char	1 Byte	-128 s.d 127	
Int	2 Byte	-32768 s.d 32767	
Short	2 Byte	-32768 s.d 32767	
Long	4 Byte	-2,147,435,648 s.d 2,147,435,647	
Float	4 Byte	3.4×10^{-38} s.d $3.4 \times 10^{+38}$	5 – 7
Double	8 Byte	1.7×10^{-308} s.d $1.7 \times 10^{+308}$	15 – 16
Long Double	10 Byte	3.4×10^{-4932} s.d $1.1 \times 10^{+4932}$	19

Gambar 6. Detail Tipe Data

Selanjutnya berikut detail dari masing masing tipe data bilangan:

a. Tipe Bilangan Bulat

Tipe Data	Ukuran (bit)	Range
Byte	8	-128 s.d. 127
Short	16	-32768 s.d. 32767
Int	32	-2147483648 s.d. 2147483647
Long	64	-9223372036854775808 s.d. 9223372036854775807

Gambar 7. Detail Tipe Bilangan Bulat (Integer)

b. Tipe bilangan Pecahan

Tipe Data	Ukuran	Jangkauan
float	32 bit, presisi 6-7 digit	$-3.4E38$ s.d $+3.4E38$
double	64 bit, presisi 14-15 digit	$-1.7E308$ s.d $1.7E308$

Gambar 8. Tipe Bilangan Pecahan

Berikut penjelasan dari masing tipe data diantaranya:

1. Character

Tipe data karakter (character data type) dalam pemrograman mengacu pada jenis data yang digunakan untuk menyimpan karakter tunggal seperti huruf, angka, tanda baca, atau simbol lainnya. Dalam berbagai bahasa pemrograman, tipe data karakter sering kali disebut sebagai "char" atau serupa. Berikut contoh cara tipe data karakter digunakan dalam bahasa pemrograman java script:

```
javascript Copy code  
  
var karakter = 'E';
```

Ketika Anda perlu menyimpan, mengelola, atau memproses data teks atau karakter, Anda memanfaatkan tipe data karakter. Tipe data karakter sangat membantu dalam pengembangan perangkat lunak untuk tugas-tugas seperti pemrosesan string, membaca dan menulis file teks, dan berbagai tindakan yang berhubungan dengan karakter dan teks.

2. Tipe Data Boolean

Tipe data boolean adalah tipe data dasar dalam pemrograman yang hanya memiliki dua nilai mungkin: true (benar) atau false (salah). Tipe data ini digunakan untuk merepresentasikan kondisi atau keadaan yang hanya memiliki dua kemungkinan, seperti hasil perbandingan, pengujian kondisi, atau variabel yang digunakan untuk mengendalikan alur program. Contoh penggunaan tipe data boolean dalam bahasa pemrograman java sebagai berikut:

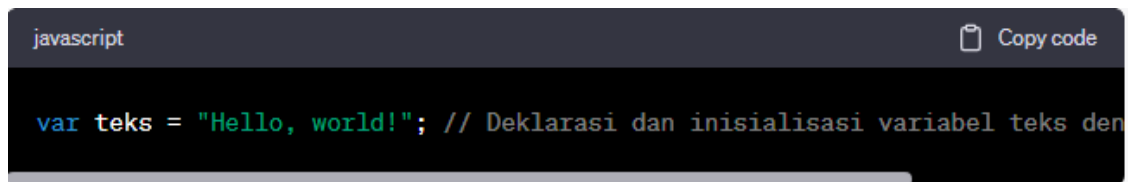
```
javascript Copy code  
  
var benar = true;  
var salah = false;
```

Tipe data boolean sangat berguna dalam pengambilan keputusan dalam program. Anda dapat menggunakan operasi perbandingan seperti "sama dengan" (==), "tidak sama dengan" (!=), "lebih besar dari" (>), "kurang dari" (<), dan lainnya untuk menghasilkan nilai boolean. Selanjutnya, nilai boolean ini dapat digunakan dalam struktur pengkondisian (misalnya, if, else if, dan else) untuk mengatur alur program berdasarkan kondisi yang diberikan.

3. Tipe Data Variabel

Variabel adalah wadah penampung data sementara dalam memori komputer yang memiliki nilai dari sebuah proses pada suatu pemrograman (Yeyi et. all, 2022). Variabel merupakan sebuah simbol atau nama yang digunakan dalam pemrograman untuk menyimpan dan merepresentasikan data dalam suatu program. Setiap variabel memiliki tipe data yang menentukan jenis data yang dapat disimpan di dalamnya, seperti angka, teks, karakter, atau nilai boolean. Variabel juga memiliki nilai yang dapat berubah selama eksekusi program. Dalam pemrograman, variabel digunakan untuk:

- a. Menyimpan Data: Variabel digunakan untuk menyimpan data yang akan digunakan atau dimanipulasi oleh program. Contohnya, Anda dapat menggunakan variabel untuk menyimpan angka, teks, atau hasil perhitungan.
- b. Memberikan Nama pada Data: Variabel memberikan nama pada data sehingga Anda dapat merujuknya dengan mudah dalam kode program. Ini membuat kode lebih mudah dibaca dan dimengerti.
- c. Mengelola State: Variabel digunakan untuk melacak keadaan (state) dalam program. Misalnya, Anda dapat menggunakan variabel untuk melacak apakah pengguna sudah masuk atau belum.

A screenshot of a code editor with a dark theme. The top bar shows 'javascript' on the left and a 'Copy code' button on the right. The main area contains a single line of code: `var teks = "Hello, world!"; // Deklarasi dan inisialisasi variabel teks den`. The text is color-coded: 'var' is blue, 'teks' is green, and the string and comment are white.

```
javascript Copy code  
  
var teks = "Hello, world!"; // Deklarasi dan inisialisasi variabel teks den
```

Tipe data variabel menentukan jenis data yang dapat disimpan di dalamnya dan operasi yang dapat dilakukan pada variabel tersebut. Hal ini membantu memastikan integritas data dan menghindari kesalahan penggunaan data yang salah dalam program Anda.

4. Konstanta

Nilai tetap yang tidak dapat dimodifikasi saat program berjalan dikenal sebagai tipe data konstan atau konstanta dalam pemrograman. Konstanta digunakan untuk menunjukkan nilai tetap yang tidak boleh berubah saat program berjalan. Konstanta dapat dinyatakan dengan berbagai cara yang mengikuti norma bahasa atau menggunakan kata kunci khusus dalam bahasa pemrograman.

```
javascript Copy code  
  
const BILANGAN_PI = 3.14159;
```

Ketika Anda memiliki nilai yang tidak boleh berubah ketika aplikasi Anda berjalan, Anda menggunakan konstanta. Sebagai hasil dari mencegah modifikasi yang tidak disengaja pada nilai-nilai penting, hal ini meningkatkan keamanan dan kejelasan kode Anda. Untuk membuat kode lebih mudah dipelihara dan dimodifikasi di masa depan, konstanta juga dapat digunakan untuk mengganti nilai yang umum digunakan dalam kode.

5. Membuat Sebuah Komentar

Dalam bahasa pemrograman, komentar adalah teks atau catatan yang disertakan ke dalam kode program untuk tujuan penjelasan atau dokumentasi. Komputer tidak akan menjalankan komentar, oleh karena itu komentar tidak berpengaruh terhadap fungsi aplikasi. Tujuan mendasar dari komentar adalah untuk membantu pengembang-baik pengembang yang menulis kode maupun pengembang lain yang mungkin akan memahami atau mengeditnya. Contoh penulisan komentar pada java:

```
javascript Copy code  
  
// Ini adalah komentar satu baris dalam JavaScript
```

```
javascript Copy code  
  
/*  
    Ini adalah  
    komentar multi-baris  
*/
```

Dalam pemrograman, komentar memainkan peran penting dalam pengaturan konteks, menjelaskan kode, dan membuatnya lebih mudah dipahami oleh diri sendiri atau pengembang lain di masa mendatang. Selain itu, komentar dapat digunakan untuk menonaktifkan sementara bagian kode yang tidak ingin Anda jalankan (juga dikenal sebagai "komentar keluar").

2.2. Penerapan Perintah Tipe Data Dasar Pada Java NetBeans

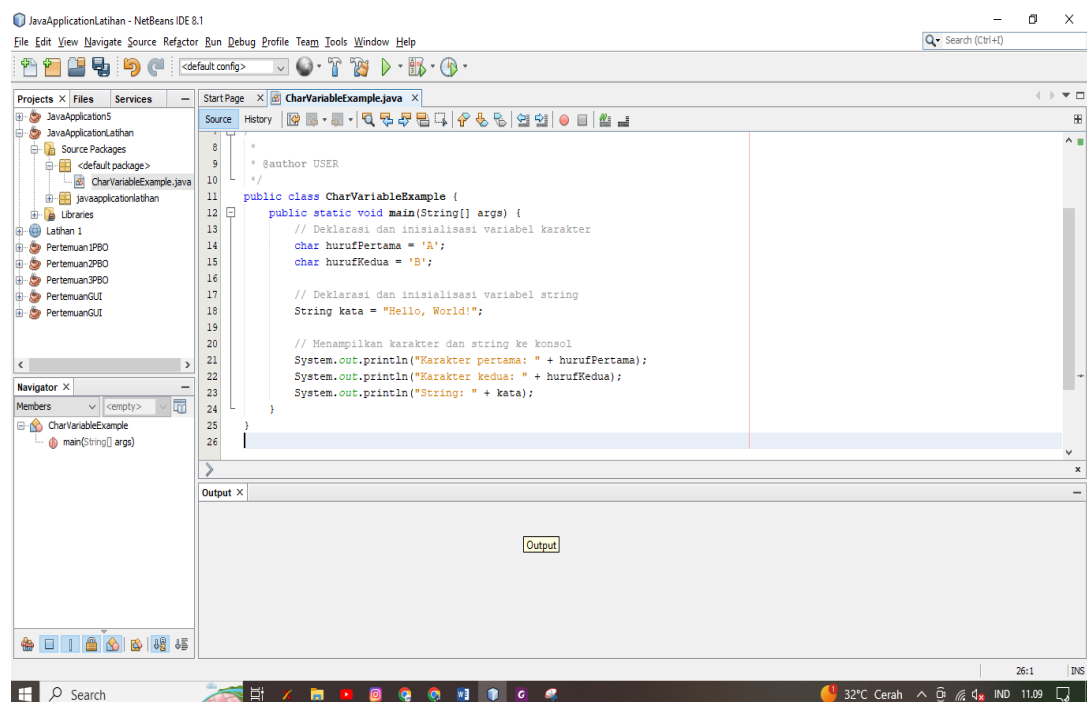
a. Penerapan tipe data Char dan Variabel pada Java NetBeans

Berikut adalah contoh penerapan tipe data karakter (char) dan variabel dalam Java menggunakan lingkungan pengembangan NetBeans. Dalam contoh ini, kita akan

mendeklarasikan variabel dengan tipe data char dan String, serta menampilkan hasilnya di konsol.

Berikut adalah langkah-langkah untuk menjalankan contoh di atas dalam lingkungan pengembangan NetBeans:

1. Buka NetBeans atau buat proyek baru jika Anda belum memiliki satu.
2. Buat kelas Java baru dengan mengklik kanan pada proyek Anda, pilih "New" (Baru), dan kemudian "Java Class" (Kelas Java). Beri nama kelas sesuai keinginan, misalnya "CharVariableExample."
3. Ketik kembali kode script ke dalam kelas yang baru dibuat.
4. Jalankan program dengan mengklik tombol "Run" (Jalankan) atau dengan menggunakan pintasan keyboard (biasanya Ctrl + F5).



Gambar 9. Implementasi Tipe Data Character dan Variabel Pada Java NetBeans Menggunakan Java Class.

Berikut Source Code class diatas:

```
public class CharVariableExample {  
    public static void main(String[] args) {  
        // Deklarasi dan inisialisasi variabel karakter  
        char hurufPertama = 'A';  
        char hurufKedua = 'B';
```

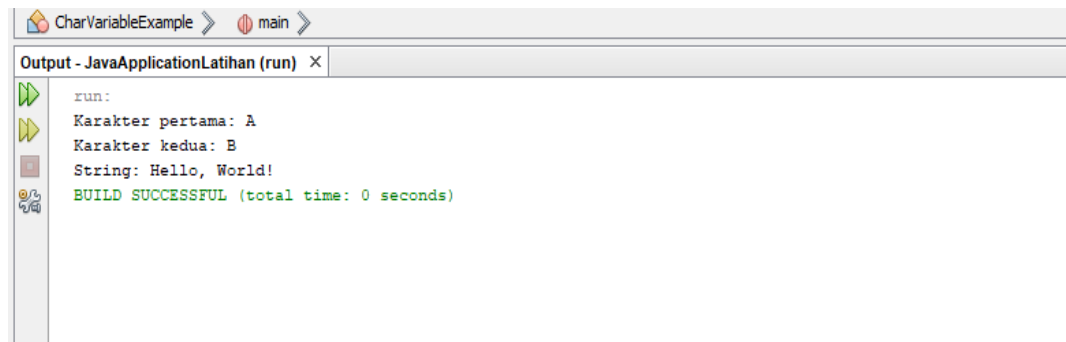
```

// Deklarasi dan inisialisasi variabel string
String kata = "Hello, World!";

// Menampilkan karakter dan string ke konsol
System.out.println("Karakter pertama: " + hurufPertama);
System.out.println("Karakter kedua: " + hurufKedua);
System.out.println("String: " + kata);
}
}

```

Hasilnya akan ditampilkan di konsol NetBeans, dan Anda akan melihat keluaran yang mencetak karakter pertama, karakter kedua, dan string ke layar konsol.



Gambar 10. Hasil Running pada layar Output

Contoh langsung ini mendemonstrasikan cara menggunakan tipe data karakter (char) dan variabel string Java. Tergantung pada kebutuhan Anda, Anda dapat menambahkan lebih banyak tindakan yang melibatkan tipe data ini atau mengubah nilai variabel sesuai keinginan Anda.

Source code di atas adalah contoh program Java yang menggunakan tipe data karakter (char) dan variabel string (String) dalam suatu kelas bernama CharVariableExample. Berikut penjelasan dari setiap bagian kode tersebut:

1. **public class CharVariableExample {**: Ini adalah deklarasi kelas Java bernama CharVariableExample. Kode program harus berada dalam kelas yang sama dengan nama file yang sama.
2. **public static void main(String[] args) {**: Ini adalah metode main, yang merupakan titik awal eksekusi program Java. Semua pernyataan dalam program akan dieksekusi dari sini.

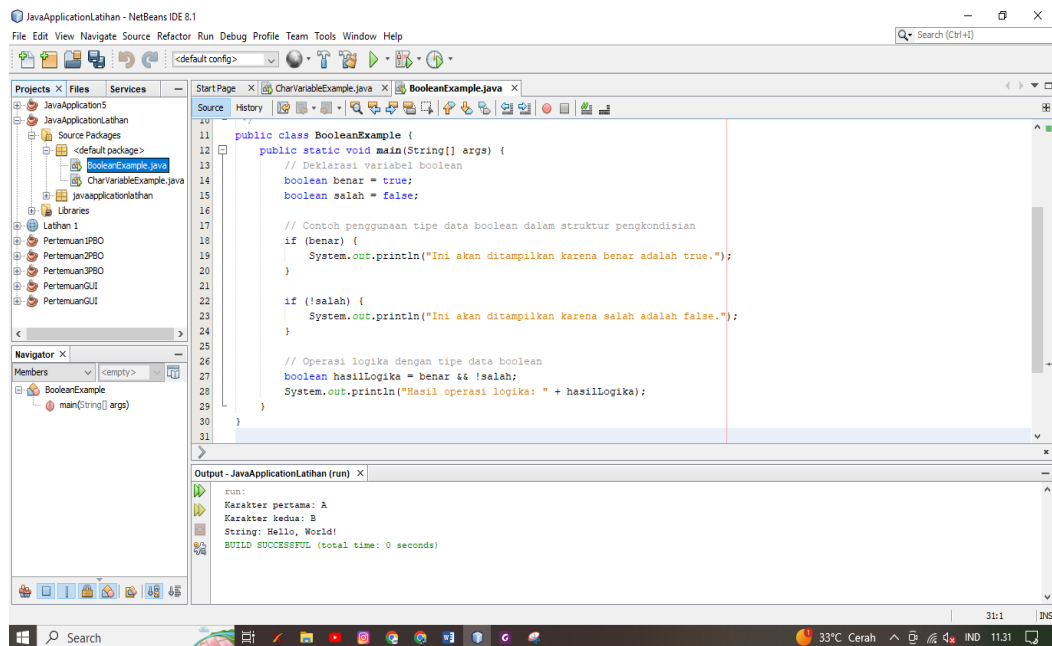
3. **char hurufPertama = 'A';** dan **char hurufKedua = 'B';**; Ini adalah deklarasi dan inisialisasi dua variabel karakter (char). Variabel **hurufPertama** diinisialisasi dengan karakter 'A', dan variabel **hurufKedua** diinisialisasi dengan karakter 'B'.
4. **String kata = "Hello, World!";**; Ini adalah deklarasi dan inisialisasi sebuah variabel string (String). Variabel kata diinisialisasi dengan string "Hello, World!".
5. **System.out.println("Karakter pertama: " + hurufPertama);**; Ini adalah pernyataan yang mencetak karakter pertama (variabel hurufPertama) ke konsol. Pernyataan ini menggunakan metode println untuk mencetak teks ke layar, dan hasilnya adalah "Karakter pertama: A".
6. **System.out.println("Karakter kedua: " + hurufKedua);**; Ini adalah pernyataan yang mencetak karakter kedua (variabel hurufKedua) ke konsol. Hasilnya adalah "Karakter kedua: B".
7. **System.out.println("String: " + kata);**; Ini adalah pernyataan yang mencetak variabel string (**kata**) ke konsol. Hasilnya adalah "String: Hello, World!".

b. Penerapan Tipe Data Boolean Pada Java NetBeans

Berikut ini adalah ilustrasi bagaimana menggunakan NetBeans Java untuk menerapkan tipe data boolean. Pada contoh ini, kita akan menggunakan tipe data boolean untuk memodifikasi alur program berdasarkan keadaan tertentu.

Berikut adalah langkah-langkah untuk menjalankan contoh tipe data boolean dalam NetBeans:

1. Buka NetBeans atau buat proyek baru jika Anda belum memiliki satu.
2. Buat kelas Java baru dengan mengklik kanan pada proyek Anda, pilih "New" (Baru), dan kemudian "Java Class" (Kelas Java). Beri nama kelas sesuai keinginan, misalnya "BooleanExample."
3. Salin kode dan tempelkannya ke dalam kelas yang baru dibuat.
4. Jalankan program dengan mengklik tombol "Run" (Jalankan) atau dengan menggunakan pintasan keyboard (biasanya Ctrl + F5).



Gambar 11. Implementasi Source Code Tipe Data Boolean Pada Java Class

Berikut Source Coding dari java class diatas :

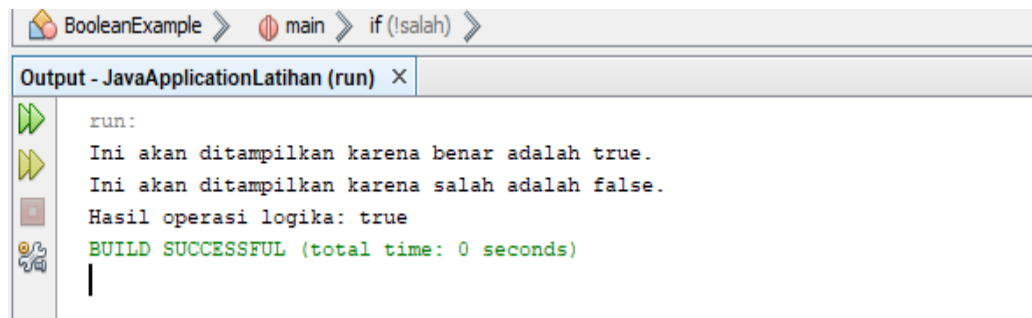
```
public class BooleanExample {
    public static void main(String[] args) {
        // Deklarasi variabel boolean
        boolean benar = true;
        boolean salah = false;

        // Contoh penggunaan tipe data boolean dalam struktur pengkondisian
        if (benar) {
            System.out.println("Ini akan ditampilkan karena benar adalah true.");
        }

        if (!salah) {
            System.out.println("Ini akan ditampilkan karena salah adalah false.");
        }

        // Operasi logika dengan tipe data boolean
        boolean hasilLogika = benar && !salah;
        System.out.println("Hasil operasi logika: " + hasilLogika);
    }
}
```

Hasilnya akan ditampilkan di konsol NetBeans. Contoh ini mendeklarasikan variabel boolean, menggunakan tipe data boolean dalam struktur pengkondisian `if`, dan melakukan operasi logika dengan tipe data boolean.



```
run:
Ini akan ditampilkan karena benar adalah true.
Ini akan ditampilkan karena salah adalah false.
Hasil operasi logika: true
BUILD SUCCESSFUL (total time: 0 seconds)
```

Gambar 12. Hasil Running Source Code Boolean pada Java di layar Output

Source code di atas adalah contoh program Java yang menggunakan tipe data boolean untuk mendeklarasikan variabel boolean, menggambarkan penggunaan tipe data boolean dalam struktur pengkondisian (if), dan melakukan operasi logika dengan tipe data boolean. Berikut adalah penjelasan dari setiap bagian kode tersebut:

1. **public class BooleanExample {**: Ini adalah deklarasi kelas Java bernama BooleanExample. Kode program harus berada dalam kelas yang sama dengan nama file yang sama.
2. **public static void main(String[] args) {**: Ini adalah metode main, yang merupakan titik awal eksekusi program Java. Semua pernyataan dalam program akan dieksekusi dari sini.
3. **boolean benar = true;** dan **boolean salah = false;**: Ini adalah deklarasi dua variabel boolean. Variabel benar diinisialisasi dengan nilai true, sementara variabel salah diinisialisasi dengan nilai false.
4. **if (benar) { ... }**: Ini adalah contoh penggunaan tipe data boolean dalam struktur pengkondisian if. Pernyataan dalam blok if akan dieksekusi jika kondisi dalam tanda kurung () bernilai true. Dalam kasus ini, pesan "Ini akan ditampilkan karena benar adalah true." akan dicetak ke konsol karena nilai benar adalah true.
5. **if (!salah) { ... }**: Ini adalah contoh penggunaan tipe data boolean dengan operator negasi (!) dalam struktur pengkondisian if. Kondisi !salah menjadi true karena salah adalah false, sehingga pesan "Ini akan ditampilkan karena salah adalah false." akan dicetak ke konsol.
6. **boolean hasilLogika = benar && !salah;**: Ini adalah contoh operasi logika dengan tipe data boolean. Variabel hasilLogika diinisialisasi dengan hasil dari

operasi logika benar && !salah. Karena benar adalah true dan !salah juga true, maka hasil operasi logika adalah true.

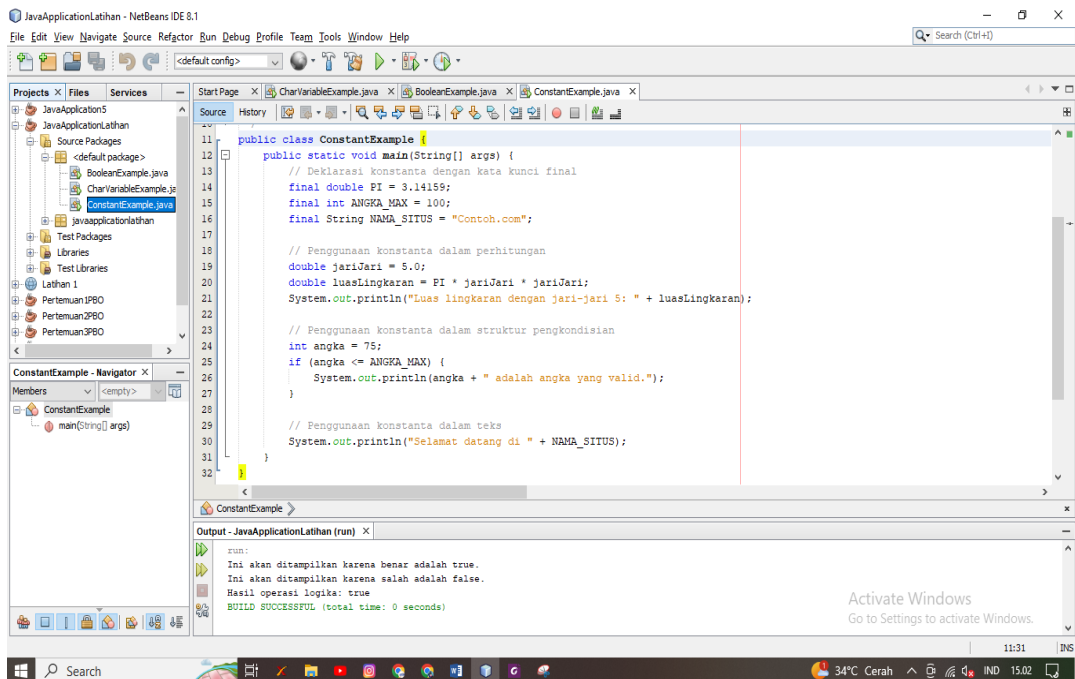
7. **System.out.println("Hasil operasi logika: " + hasilLogika);**: Ini adalah pernyataan yang mencetak hasil operasi logika (hasilLogika) ke konsol. Hasilnya akan mencetak "Hasil operasi logika: true."

c. Penerapan Tipe Data Konstanta Pada Java NetBeans

Dalam Java, Anda dapat menggunakan kata kunci final untuk mendeklarasikan konstanta. Berikut adalah contoh penerapan tipe data konstanta dalam Java menggunakan NetBeans:

Berikut adalah langkah-langkah untuk menjalankan contoh di atas dalam lingkungan pengembangan NetBeans:

1. Buka NetBeans atau buat proyek baru jika Anda belum memiliki satu.
2. Buat kelas Java baru dengan mengklik kanan pada proyek Anda, pilih "New" (Baru), dan kemudian "Java Class" (Kelas Java). Beri nama kelas sesuai keinginan, misalnya "ConstantExample."
3. Salin source code dan tempelkannya ke dalam kelas yang baru dibuat.
4. Jalankan program dengan mengklik tombol "Run" (Jalankan) atau dengan menggunakan pintasan keyboard (biasanya Ctrl + F5).



Gambar 13. Implementasi Sorce Code Tipe Data Konstanta pada Java NetBeans
Berikut Source Code dari gambar diatas:


```

Public class ConstantExample {
    public static void main(String[] args) {
        // Deklarasi konstanta dengan kata kunci final
        final double PI = 3.14159;
        final int ANGKA_MAX = 100;
        final String NAMA_SITUS = "Contoh.com";

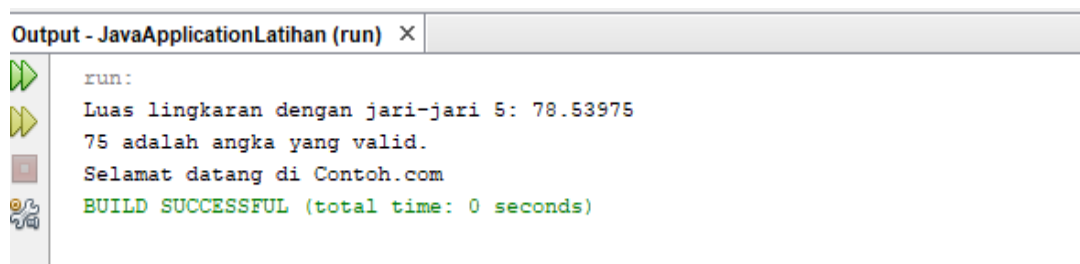
        // Penggunaan konstanta dalam perhitungan
        double jariJari = 5.0;
        double luasLingkaran = PI * jariJari * jariJari;
        System.out.println("Luas lingkaran dengan jari-jari 5: " + luasLingkaran);

        // Penggunaan konstanta dalam struktur pengkondisian
        int angka = 75;
        if (angka <= ANGKA_MAX) {
            System.out.println(angka + " adalah angka yang valid.");
        }

        // Penggunaan konstanta dalam teks
        System.out.println("Selamat datang di " + NAMA_SITUS);
    }
}

```

Hasilnya akan ditampilkan di konsol NetBeans. Contoh ini mendeklarasikan konstanta dengan kata kunci final, dan kemudian menggunakannya dalam perhitungan matematika, struktur pengkondisian, dan teks. Variabel-variabel konstan ini memiliki nilai yang tidak dapat diubah selama eksekusi program dan dapat digunakan untuk merepresentasikan nilai-nilai yang bersifat tetap.



Gambar 14. Hasil Running Source Code Konstanta pada Java di layar Output

Source code di atas adalah contoh program Java yang mendemonstrasikan penggunaan tipe data konstanta (final) dalam Java. Berikut adalah penjelasan dari setiap bagian kode tersebut:

1. **public class ConstantExample {**: Ini adalah deklarasi kelas Java yang dinamakan ConstantExample. Kode program harus berada dalam kelas yang sama dengan nama file yang sama.

2. **public static void main(String[] args) {**: Ini adalah metode main, yang merupakan titik awal eksekusi program Java. Semua pernyataan dalam program akan dieksekusi dari sini.
3. **final double PI = 3.14159;; final int ANGKA_MAX = 100;;** dan **final String NAMA_SITUS = "Contoh.com";**: Ini adalah deklarasi konstanta dengan kata kunci final. Konstanta ini dideklarasikan dengan tipe data yang sesuai dan memiliki nilai tetap yang tidak dapat diubah selama eksekusi program. Contoh konstanta yang dideklarasikan di sini adalah PI (nilai π), ANGKA_MAX (angka maksimum), dan NAMA_SITUS (nama situs web).
4. **double jariJari = 5.0;;**: Ini adalah deklarasi dan inisialisasi variabel jariJari yang digunakan dalam perhitungan untuk menghitung luas lingkaran.
5. **double luasLingkaran = PI * jariJari * jariJari;;**: Ini adalah perhitungan yang menggunakan konstanta PI dan variabel jariJari untuk menghitung luas lingkaran. Hasilnya disimpan dalam variabel luasLingkaran.
6. **System.out.println("Luas lingkaran dengan jari-jari 5: " + luasLingkaran);**: Ini adalah pernyataan yang mencetak hasil perhitungan (luas lingkaran) ke konsol. Hasilnya akan mencetak pesan "Luas lingkaran dengan jari-jari 5: ..." dengan nilai luas yang dihitung.
7. **int angka = 75;;**: Ini adalah deklarasi dan inisialisasi variabel angka yang digunakan dalam struktur pengkondisian.
8. **if (angka <= ANGKA_MAX) { ... }**: Ini adalah contoh penggunaan konstanta ANGKA_MAX dalam struktur pengkondisian if. Pernyataan dalam blok if akan dieksekusi jika nilai angka kurang dari atau sama dengan nilai konstanta ANGKA_MAX. Dalam contoh ini, pesan akan dicetak jika kondisinya benar.
9. **System.out.println("Selamat datang di " + NAMA_SITUS);**: Ini adalah pernyataan yang mencetak teks yang menggabungkan konstanta NAMA_SITUS ke konsol. Hasilnya akan mencetak pesan "Selamat datang di Contoh.com".

BAB III

TEORI & KONSEP PEMROGRAMAN BERBASIS OBJECT

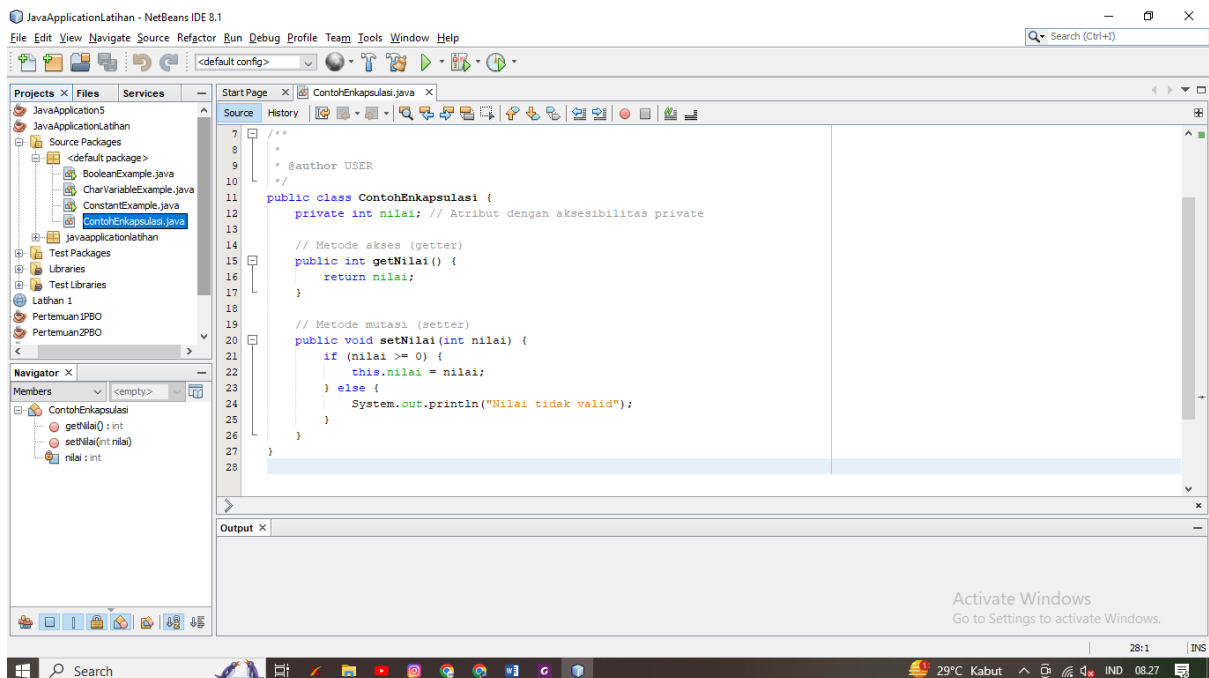
3.1. Penerapan Karakteristik Pemrograman Berorientasi Object Pada Java

Berikut eneraan karakteristik pemrograman berorientasi Object diantaranya ada Enkapsulasi, Pewarisan atau Inheritance dan Polymorphism.

1. Enkapsulasi

Salah satu ide dasar pemrograman berorientasi objek (OOP) adalah enkapsulasi, yang merupakan proses membungkus data dan metode di dalam sebuah objek sehingga hanya objek tersebut yang dapat mengakses dan memodifikasi data. Tujuan utama enkapsulasi adalah untuk menyembunyikan detail implementasi inti dari sebuah objek dan memungkinkan akses yang terkontrol dengan menggunakan metode yang telah ditentukan.

Enkapsulasi membatasi tingkat aksesibilitas untuk data (sering disebut sebagai karakteristik atau properti) yang terkandung dalam suatu objek. Ini berarti bahwa hanya metode yang telah ditentukan - sering disebut sebagai metode akses (getter) dan metode mutasi (setter) - yang dapat digunakan untuk mendapatkan atau mengubah data. Berlawanan dengan metode mutasi, yang digunakan untuk mengubah nilai atribut, metode akses digunakan untuk membaca nilai atribut. Beginilah cara enkapsulasi membantu dalam memelihara. Berikut contoh script Enkapsulasi:



Gambar 15. Contoh Enkapsulasi

Source Code Sample Enkapsulasi:

```
public class ContohEnkapsulasi {  
    private int nilai; // Atribut dengan aksesibilitas private  
    // Metode akses (getter)  
    public int getNilai() {  
        return nilai;  
    }  
    // Metode mutasi (setter)  
    public void setNilai(int nilai) {  
        if (nilai >= 0) {  
            this.nilai = nilai;  
        } else {  
            System.out.println("Nilai tidak valid");  
        }  
    }  
}
```

Dalam contoh di atas, atribut **nilai** dideklarasikan sebagai private, sehingga hanya dapat diakses melalui metode akses **getNilai()** dan diubah melalui metode mutasi **setNilai()**. Hal ini memberikan kontrol lebih besar atas bagaimana data dapat dimanipulasi dan memungkinkan implementasi yang lebih aman dan terstruktur dalam pemrograman berorientasi objek.

Berikut Penjelasan Sorce Coding Enkapsulasi diatas:

a) public class ContohEnkapsulasi {

Ini adalah deklarasi kelas dengan nama "ContohEnkapsulasi". Kelas ini adalah wadah untuk atribut dan metode yang terkait.

b) private int nilai;

Ini adalah deklarasi atribut dengan nama "nilai" yang memiliki tipe data int (integer). Atribut ini ditandai dengan aksesibilitas private, yang berarti atribut ini hanya dapat diakses secara langsung dari dalam kelas ini dan tidak bisa diakses dari luar kelas.

c) public int getNilai() {

Ini adalah metode akses (getter) yang digunakan untuk mengembalikan nilai atribut "nilai". Metode ini memiliki tipe data pengembalian int.

d) return nilai;

Pada metode `getNilai()`, nilai dari atribut "nilai" dikembalikan kepada pemanggil metode ini.

e) `public void setNilai(int nilai) {`

Ini adalah metode mutasi (setter) yang digunakan untuk mengubah nilai atribut "nilai". Metode ini memiliki parameter berupa nilai integer yang akan digunakan untuk mengganti nilai atribut.

f) `if (nilai >= 0) {`

Pada metode `setNilai()`, terdapat pengecekan apakah nilai yang ingin diubah adalah nilai positif atau nol.

g) `this.nilai = nilai;`

Jika nilai yang diberikan adalah non-negatif, maka nilai atribut "nilai" akan diubah dengan nilai baru yang diberikan melalui parameter.

h) `System.out.println("Nilai tidak valid");`

Jika nilai yang diberikan kurang dari nol, maka pesan "Nilai tidak valid" akan dicetak ke layar.

Dengan mengembangkan metode akses (getter) dan metode mutasi (setter) untuk mengakses dan mengubah nilai atribut dengan aman, kelas "Contoh Enkapsulasi" menyelesaikan enkapsulasi untuk atribut "nilai". Metode-metode ini adalah satu-satunya cara untuk mengakses dan mengubah properti "value", yang memiliki akses pribadi. Hal ini memberikan Anda kontrol yang lebih baik terhadap data dan implementasi pemrograman berorientasi objek yang lebih terorganisir.

2. Pewarisan (Inheritance)

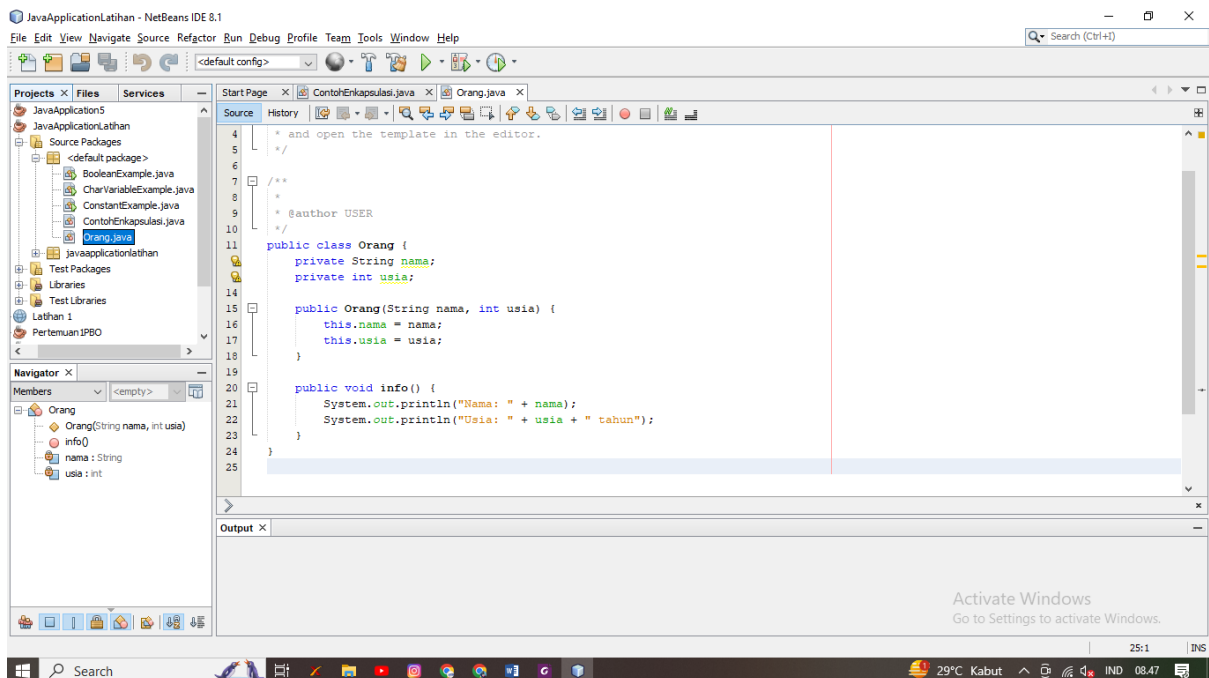
Pewarisan (Inheritance) adalah salah satu konsep utama dalam pemrograman berorientasi objek (OOP) yang memungkinkan sebuah kelas untuk mewarisi sifat dan perilaku dari kelas lain. Konsep ini memungkinkan penciptaan hierarki kelas, di mana sebuah kelas yang baru dapat dibangun berdasarkan kelas yang sudah ada, mengambil karakteristik atau atribut serta metode yang telah didefinisikan dalam kelas yang sudah ada. Ada beberapa konsep penting terkait pewarisan:

- a. **Kelas Induk (Superclass atau Parent Class):** Kelas yang sudah ada dan memberikan karakteristik (atribut dan metode) kepada kelas anak. Kelas ini juga disebut kelas super atau kelas induk.

- b. **Kelas Anak (Subclass atau Child Class):** Kelas yang dibangun berdasarkan kelas induk. Kelas anak mewarisi atribut dan metode dari kelas induk, dan juga dapat menambahkan atau mengubah perilaku tersebut.
- c. **Pewarisan Atribut:** Kelas anak mewarisi atribut (variabel) dari kelas induk. Ini berarti bahwa atribut yang sudah ada dalam kelas induk juga dapat digunakan dalam kelas anak.
- d. **Pewarisan Metode:** Kelas anak mewarisi metode (fungsi) dari kelas induk. Metode ini dapat digunakan dalam kelas anak, dan kelas anak juga dapat menambahkan metode tambahan atau mengganti (override) metode yang sudah ada.
- e. **Override (Penggantian):** Kelas anak dapat mengganti (override) metode yang sudah ada dalam kelas induk dengan implementasi yang sesuai untuk kelas anak tersebut. Ini memungkinkan perilaku yang berbeda antara kelas anak dan kelas induk.

Menggunakan kembali kode, meningkatkan hirarki kelas, dan memfasilitasi polimorfisme-kemampuan objek kelas anak untuk digunakan sebagai objek kelas induk-semuanya dimungkinkan melalui pewarisan. Selain mengurangi duplikasi kode dan meningkatkan pemahaman tentang hubungan antar kelas dalam program, hal ini memungkinkan pengembang untuk mengatur dan mengelola kode dengan lebih efektif. Berikut contoh script java dalam penggunaan code Pewarisan (Inheritance) :

Class Induk :

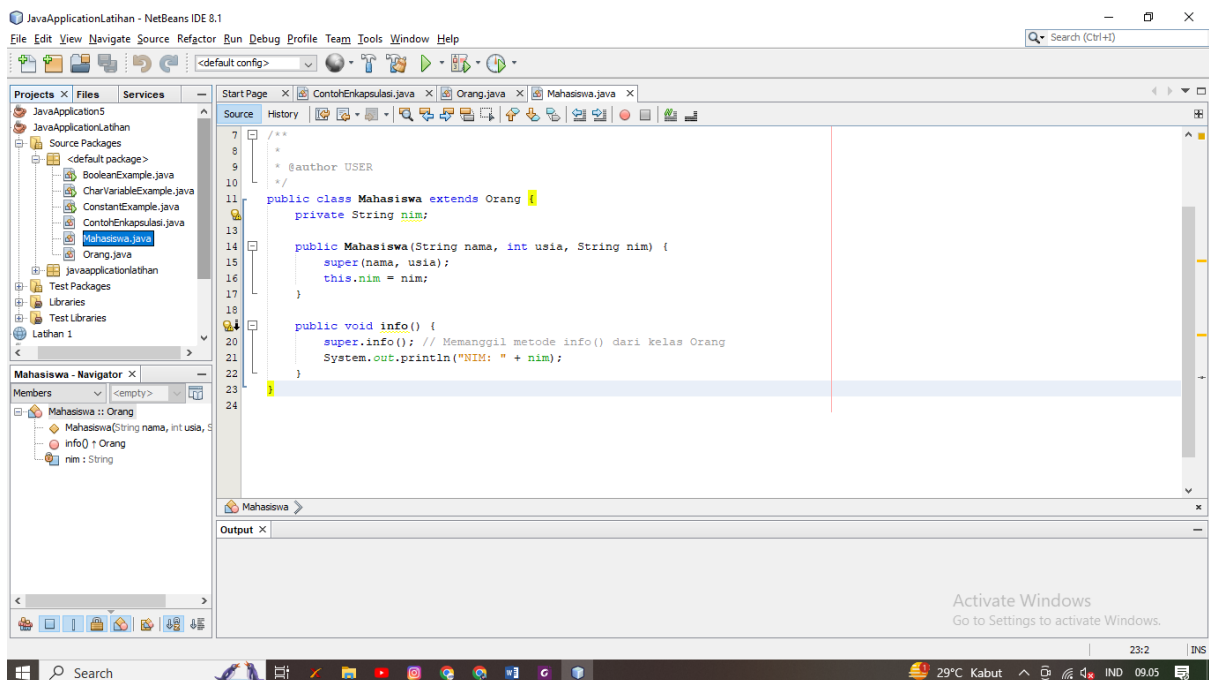


Gambar 16. Contoh Script Pewarisan (Class Induk)

Source Code :

```
public class Orang {  
    private String nama;  
    private int usia;  
    public Orang(String nama, int usia) {  
        this.nama = nama;  
        this.usia = usia;  
    }  
    public void info() {  
        System.out.println("Nama: " + nama);  
        System.out.println("Usia: " + usia + " tahun");  
    }  
}
```

Berikut Class “Mahasiswa” Class yang mewarisi class induk:



Gambar 17. Class Pewarisan

Source Code dari gambar 17 sebagai berikut:

```
public class Mahasiswa extends Orang {  
    private String nim;  
    public Mahasiswa(String nama, int usia, String nim) {  
        super(nama, usia);  
    }  
}
```

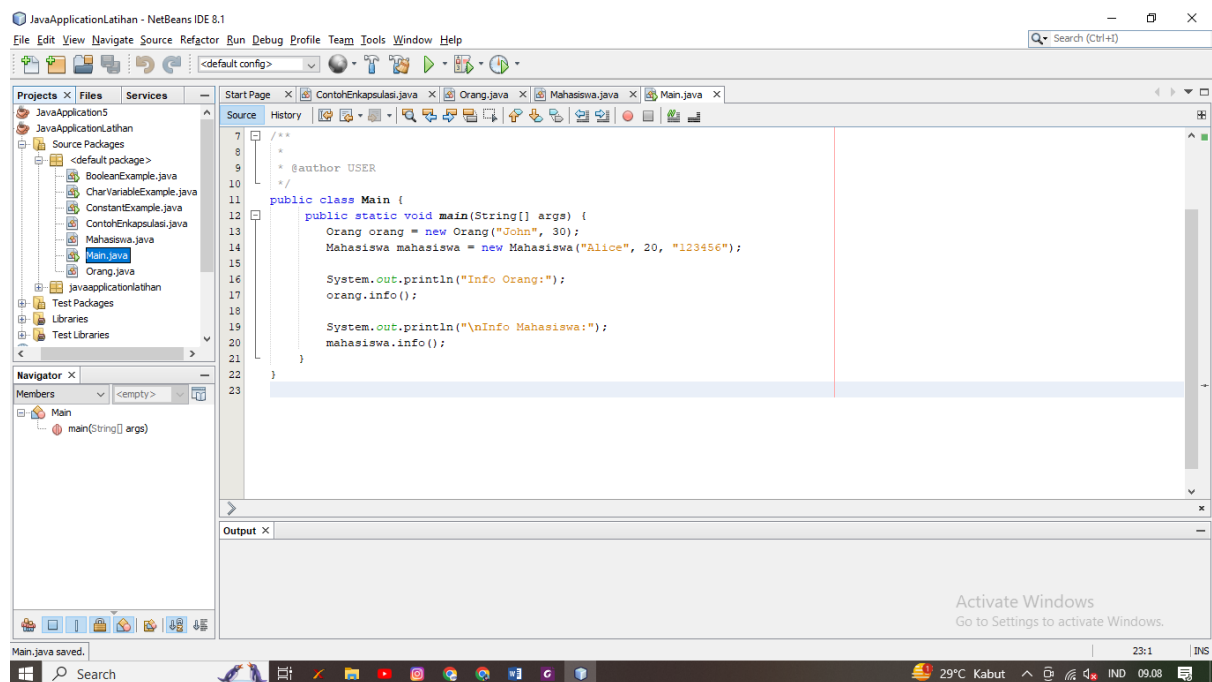
```

        this.nim = nim;
    }

    public void info() {
        super.info(); // Memanggil metode info() dari kelas Orang
        System.out.println("NIM: " + nim);
    }
}

```

Kemudian, buat kelas Main sebagai pengujian pewarisan :



Gambar 18. Class Main Pewarisan

Setelah Anda menambahkan semua kelas di atas, Anda dapat menjalankan program Anda. Hasilnya akan menampilkan informasi tentang objek **Orang** dan **Mahasiswa**. Perhatikan bahwa kelas **Mahasiswa** mewarisi metode **info()** dari kelas **Orang**, tetapi juga memiliki implementasi sendiri yang memanggil **super.info()** untuk mencetak informasi kelas **Orang** sebelum menambahkan informasi spesifik mahasiswa.

3. Polymorphism

Salah satu ide dasar pemrograman berorientasi objek (OOP) adalah polimorfisme, yang menggambarkan kapasitas objek untuk mengadopsi berbagai bentuk atau bertindak dalam berbagai situasi. Dengan kata lain, polimorfisme memungkinkan objek dari berbagai kelas bereaksi terhadap metode dengan cara yang sesuai untuk setiap jenis.

Ada dua jenis polimorfisme dalam OOP:

a. Polimorfisme Kompilasi (Compile-time Polymorphism atau Static Polymorphism):

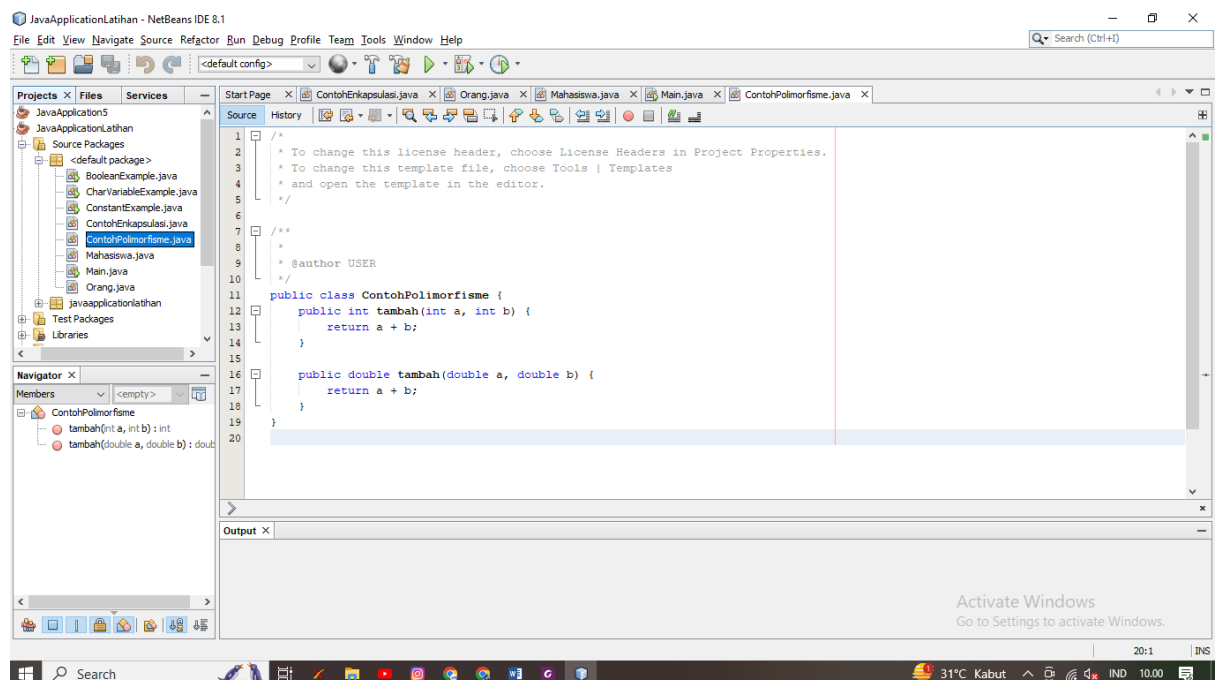
Polimorfisme kompilasi terjadi saat pengikatan metode terjadi selama kompilasi program.

Ini juga dikenal sebagai "metode overloading."

Metode overloading terjadi ketika dua atau lebih metode dalam sebuah kelas memiliki nama yang sama tetapi memiliki parameter yang berbeda.

Pemilihan metode yang akan dipanggil tergantung pada tipe parameter saat pemanggilan metode dilakukan.

Contoh polimorfisme kompilasi (overloading) dalam Java:



Gambar19. Contoh Polimorfisme

Source Code:

```
public class ContohPolimorfisme {  
    public int tambah(int a, int b) {  
        return a + b;  
    }  
    public double tambah(double a, double b) {  
        return a + b;  
    }  
}
```

b. Polimorfisme Runtime (Runtime Polymorphism atau Dynamic Polymorphism):

Polimorfisme runtime terjadi saat pengikatan metode terjadi selama eksekusi program. Ini juga dikenal sebagai "metode overriding."

Metode overriding terjadi ketika sebuah kelas anak (subclass) memberikan implementasi yang sesuai untuk metode yang telah didefinisikan dalam kelas induk (superclass).

Pemilihan metode yang akan dipanggil tergantung pada objek yang digunakan pada saat runtime.

Contoh polimorfisme runtime (overriding) dalam Java:

```
class Hewan {
    void suara() {
        System.out.println("Beberapa hewan bersuara.");
    }
}
class Kucing extends Hewan {
    void suara() {
        System.out.println("Kucing bersuara: Meong!");
    }
}
public class Main {
    public static void main(String[] args) {
        Hewan hewan1 = new Hewan();
        Hewan hewan2 = new Kucing();
        hewan1.suara(); // Output: Beberapa hewan bersuara.
        hewan2.suara(); // Output: Kucing bersuara: Meong!
    }
}
```

Dalam contoh di atas, metode **suara()** di kelas **Kucing** melakukan overriding terhadap metode yang sama di kelas **Hewan**. Hasil pemanggilan metode **suara()** akan bergantung pada tipe objek yang digunakan pada saat runtime.

Polimorfisme memungkinkan fleksibilitas dan modularitas dalam desain program, memungkinkan penggunaan kode yang sama untuk berbagai jenis objek, dan mendukung konsep seperti penggunaan antarmuka (interfaces) dan pewarisan (inheritance) dalam OOP.

BAB IV

OPERATOR PADA BAHASA JAVA

4.1.Operator Aritmatika Pada Java

Dalam Java, operator aritmatika adalah simbol-simbol yang digunakan untuk melakukan operasi matematika pada bilangan. Berikut adalah beberapa operator aritmatika yang umum digunakan dalam Java:

1. Penambahan (+): Operator ini digunakan untuk menambahkan dua nilai. Contoh:

```
java Copy code  
  
int hasil = 5 + 3; // Hasil akan menjadi 8
```

2. Pengurangan (-): Operator ini digunakan untuk mengurangi dua nilai. Contoh:

```
java Copy code  
  
int hasil = 10 - 4; // Hasil akan menjadi 6
```

3. Perkalian (*): Operator ini digunakan untuk mengalikan dua nilai. Contoh:

```
java Copy code  
  
int hasil = 6 * 5; // Hasil akan menjadi 30
```

4. Pembagian (/): Operator ini digunakan untuk membagi dua nilai. Ini menghasilkan nilai desimal jika salah satu atau kedua operan adalah desimal. Contoh:

```
java Copy code  
  
double hasil = 10.0 / 3.0; // Hasil akan menjadi 3.3333333333333335
```

5. Modulus (%): Operator ini mengembalikan sisa dari hasil pembagian dua nilai. Contoh:

```
java Copy code  
  
int hasil = 10 % 3; // Hasil akan menjadi 1, karena 10 dibagi 3 akan mengha
```

6. Inkremen (++) dan Dekremen (--): Operator ini digunakan untuk meningkatkan atau mengurangi nilai satu unit.

```
java Copy code  
  
int angka = 5;  
angka++; // Setelah ini, nilai angka akan menjadi 6  
angka--; // Setelah ini, nilai angka akan kembali menjadi 5
```

Operator aritmatika ini digunakan dalam berbagai operasi matematika dalam pemrograman untuk melakukan perhitungan dan manipulasi nilai numerik.

4.2.Operator Pemberi Nilai

Operator pemberi nilai dalam Java adalah operator yang digunakan untuk menginisialisasi atau mengganti nilai dari suatu variabel. Operator ini juga dikenal sebagai operator assignment. Operator pemberi nilai umumnya digunakan untuk memberikan nilai kepada variabel.

Di Java, operator pemberi nilai utama adalah operator "=" (sama dengan). Dengan operator ini, Anda dapat menginisialisasi variabel dengan nilai tertentu atau mengganti nilai variabel yang sudah ada.

Berikut adalah beberapa contoh penggunaan operator pemberi nilai dalam Java:

1. Menginisialisasi Variabel:

```
int angka = 10; // Menginisialisasi variabel "angka" dengan nilai 10
```

2. Mengganti Nilai Variabel:

```
int nilai = 5; // Menginisialisasi variabel "nilai" dengan nilai 5
```

```
nilai = 8; // Mengganti nilai variabel "nilai" dengan 8
```

3. Operator Pemberi Nilai Gabungan (Compound Assignment):

Java juga menyediakan operator pemberi nilai gabungan yang memungkinkan Anda untuk melakukan operasi aritmatika dan kemudian memberikan hasilnya ke variabel yang sama.

```
int jumlah = 10;
```

```
jumlah += 5; // Mengganti nilai "jumlah" dengan hasil penambahan (jumlah = jumlah + 5)
```

Operator pemberi nilai gabungan lainnya meliputi -= (pengurangan), *= (perkalian), /= (pembagian), dan lain-lain.

Operator pemberi nilai adalah bagian penting dari pemrograman karena memungkinkan Anda untuk mengelola dan memanipulasi data dalam program Anda dengan mengubah nilai variabel sesuai kebutuhan.

4.3. Operator Penambah dan Pengurang

Operator penambah dan pengurang dalam Java adalah operator yang digunakan untuk meningkatkan (inkrement) atau mengurangi (dekrement) nilai variabel bilangan bulat (seperti `int`) sebanyak satu. Operator penambah menambahkan satu ke nilai variabel, sedangkan operator pengurang mengurangi satu dari nilai variabel. Operator ini sering digunakan dalam loop atau dalam situasi di mana Anda perlu mengubah nilai variabel dengan jumlah tertentu. Dalam Java, ada dua operator penambah dan pengurang yang umum digunakan:

1. ****Operator Penambah (++)**:** Operator ini digunakan untuk menambah satu ke nilai variabel. Anda dapat menggunakannya sebagai postfix (ditempatkan setelah variabel) atau prefix (ditempatkan sebelum variabel). Contoh penggunaan operator penambah sebagai postfix:

```
java Copy code  
  
int angka = 5;  
angka++; // Nilai angka akan menjadi 6 setelah pernyataan ini dieksekusi
```

Contoh penggunaan operator penambah sebagai prefix:

```
java Copy code  
  
int angka = 5;  
++angka; // Nilai angka juga akan menjadi 6 setelah pernyataan ini dieksekusi
```

2. **Operator Pengurang (--):** Operator ini digunakan untuk mengurangi satu dari nilai variabel. Seperti operator penambah, Anda juga dapat menggunakannya sebagai postfix atau prefix. Contoh penggunaan operator pengurang sebagai postfix:

```
java Copy code  
  
int angka = 5;  
angka--; // Nilai angka akan menjadi 4 setelah pernyataan ini dieksekusi
```

Contoh penggunaan operator pengurang sebagai prefix:

```
java Copy code  
  
int angka = 5;  
--angka; // Nilai angka juga akan menjadi 4 setelah pernyataan ini dieksekusi
```

Operator penambah dan pengurang sering digunakan dalam perulangan (looping) untuk mengatur indeks atau perhitungan. Mereka juga dapat digunakan dalam ekspresi matematika yang melibatkan peningkatan atau pengurangan nilai variabel.

4.4. Operator Pembandingan

Operator pembandingan dalam Java adalah operator yang digunakan untuk membandingkan dua nilai atau ekspresi. Operator-operator ini membandingkan nilai-nilai tersebut dan menghasilkan nilai boolean (true atau false) sebagai hasilnya, yang menunjukkan apakah perbandingan tersebut benar atau salah. Operator pembandingan digunakan dalam pengambilan keputusan dan pengontrolan alur program. Berikut adalah beberapa operator pembandingan yang umum digunakan dalam Java:

1. **== (sama dengan):** Memeriksa apakah dua nilai atau ekspresi sama. Contoh:

```
int angka1 = 5;
int angka2 = 5;
boolean hasil = (angka1 == angka2); // Hasilnya akan true karena angka1 sama dengan
angka2
```

2. **!= (tidak sama dengan):** Memeriksa apakah dua nilai atau ekspresi tidak sama. Contoh:

```
int angka1 = 5;
int angka2 = 10;
boolean hasil = (angka1 != angka2); // Hasilnya akan true karena angka1 tidak sama
dengan angka2
```

3. **< (kurang dari):** Memeriksa apakah nilai di sebelah kiri lebih kecil dari nilai di sebelah kanan. Contoh:

```
int angka1 = 5;
int angka2 = 10;
boolean hasil = (angka1 < angka2); // Hasilnya akan true karena angka1 kurang dari
angka2
```

4. **> (lebih dari):** Memeriksa apakah nilai di sebelah kiri lebih besar dari nilai di sebelah kanan. Contoh:

```
int angka1 = 10;
int angka2 = 5;
boolean hasil = (angka1 > angka2); // Hasilnya akan true karena angka1 lebih besar dari
angka2
```

5. **<= (kurang dari atau sama dengan):** Memeriksa apakah nilai di sebelah kiri lebih kecil dari atau sama dengan nilai di sebelah kanan. Contoh:

```
int angka1 = 5;
```

```
int angka2 = 5;
```

```
boolean hasil = (angka1 <= angka2); // Hasilnya akan true karena angka1 kurang dari atau sama dengan angka2
```

6. **>= (lebih dari atau sama dengan):** Memeriksa apakah nilai di sebelah kiri lebih besar dari atau sama dengan nilai di sebelah kanan. Contoh:

```
int angka1 = 10;
```

```
int angka2 = 5;
```

```
boolean hasil = (angka1 >= angka2); // Hasilnya akan true karena angka1 lebih besar dari atau sama dengan angka2
```

Operator-operator pembandingan ini digunakan dalam pernyataan kondisional (seperti if, else if, dan switch) untuk membuat keputusan berdasarkan hasil perbandingan nilai-nilai yang berbeda. Hasil dari operator pembandingan adalah nilai boolean yang menentukan apakah perbandingan tersebut benar (true) atau salah (false).

4.5. Operator Logika

Operator logika dalam Java adalah operator yang digunakan untuk menggabungkan atau memanipulasi hasil dari ekspresi logika (perbandingan) untuk menghasilkan nilai boolean (true atau false). Operator-operator logika digunakan untuk membuat keputusan berdasarkan beberapa kondisi yang saling berkaitan. Berikut adalah beberapa operator logika yang umum digunakan dalam Java:

1. **&& (AND Logika):** Operator ini mengembalikan true jika kedua ekspresi yang diperiksa adalah true. Jika salah satu atau kedua ekspresi adalah false, maka hasilnya akan false.

Contoh:

```
boolean x = true;
```

```
boolean y = false;
```

```
boolean hasil = x && y; // Hasilnya akan false karena x AND y adalah false
```

2. **|| (OR Logika):** Operator ini mengembalikan true jika salah satu dari dua ekspresi yang diperiksa adalah true. Jika kedua ekspresi adalah false, maka hasilnya akan false. Contoh:

```
boolean x = true;
```

```
boolean y = false;
```

```
boolean hasil = x || y; // Hasilnya akan true karena x OR y adalah true
```

3. **! (NOT Logika):** Operator ini mengganti nilai boolean menjadi sebaliknya. Jika suatu ekspresi adalah true, maka dengan operator ! akan menjadi false, dan sebaliknya. Contoh:
- ```
boolean x = true;
boolean hasil = !x; // Hasilnya akan false karena NOT x adalah false
```

Operator logika digunakan dalam pernyataan kondisional (seperti if, else, dan while) dan ekspresi logika yang kompleks untuk mengontrol alur program. Dengan operator logika, Anda dapat membuat keputusan yang kompleks berdasarkan berbagai kondisi yang bergantung satu sama lain.



## BAB V

### PERINTAH MASUKAN, PENYELEKSIAN KONDISI, PERULANGAN DAN ARRAY PADA PEMROGRAMAN JAVA

#### 5.1. Perintah Masukan Pada Program Java

Perintah masukan (input) dalam Java mengacu pada cara Anda mengambil data atau informasi dari pengguna atau sumber lainnya ke dalam program Anda. Dalam Java, Anda dapat menggunakan berbagai metode dan kelas untuk mengambil masukan dari pengguna. Dua kelas yang umum digunakan untuk mengambil masukan pengguna adalah **Scanner** dan **BufferedReader**.

Berikut adalah contoh penggunaan **Scanner** untuk mengambil masukan pengguna dari keyboard:



```
7 /**
8 *
9 * @author USER
10 */
11 import java.util.Scanner;
12
13 public class InputExample {
14 public static void main(String[] args) {
15 Scanner scanner = new Scanner(System.in); // Membuat objek Scanner
16
17 System.out.print("Masukkan nama Anda: ");
18 String nama = scanner.nextLine(); // Membaca sebuah baris masukan sebagai string
19
20 System.out.print("Masukkan umur Anda: ");
21 int umur = scanner.nextInt(); // Membaca sebuah bilangan bulat
22
23 System.out.println("Halo, " + nama + "! Anda berusia " + umur + " tahun.");
24
25 scanner.close(); // Menutup objek Scanner
26 }
27 }
```

Gambar 20. Contoh Script Perintah Masukkan

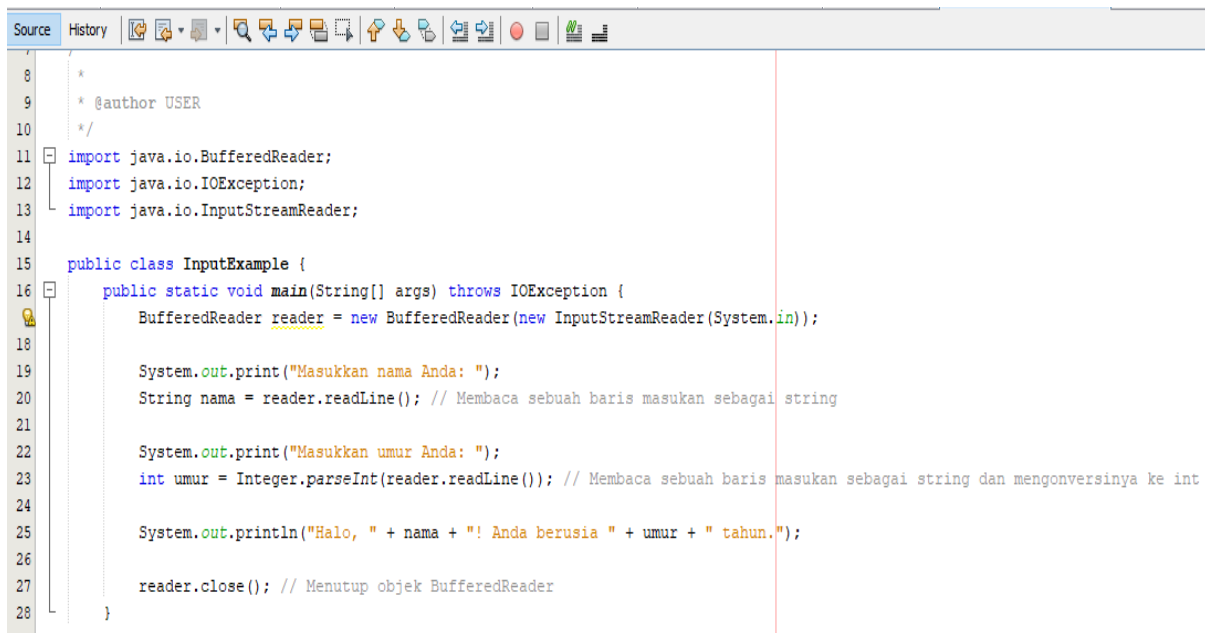
Di atas, kita mengimpor kelas **Scanner** dan kemudian membuat objek **Scanner** untuk mengambil masukan dari pengguna melalui **System.in** (keyboard). Selanjutnya, kita menggunakan metode **nextLine()** untuk membaca sebuah baris masukan sebagai string dan **nextInt()** untuk membaca bilangan bulat.

Berikut penjelasan Script pada Gambar 20:

1. `import java.util.Scanner;`: Ini adalah pernyataan impor yang mengimpor kelas `Scanner` dari paket `java.util`, yang digunakan untuk membaca masukan dari pengguna.
2. `public class InputExample {`: Ini adalah deklarasi kelas Java utama yang bernama `InputExample`.
3. `public static void main(String[] args) {`: Ini adalah metode `main` yang merupakan titik masuk program. Metode ini akan dieksekusi ketika program dijalankan.
4. `Scanner scanner = new Scanner(System.in);`: Ini adalah pembuatan objek `Scanner` yang digunakan untuk membaca masukan dari keyboard. Objek ini dibuat dengan menggunakan `System.in` sebagai sumber masukan.
5. `System.out.print("Masukkan nama Anda: ");`: Ini adalah pernyataan cetak yang meminta pengguna untuk memasukkan nama mereka.
6. `String nama = scanner.nextLine();`: Ini adalah pernyataan yang membaca sebuah baris masukan sebagai string yang dimasukkan oleh pengguna dan menyimpannya dalam variabel `nama`.
7. `System.out.print("Masukkan umur Anda: ");`: Ini adalah pernyataan cetak yang meminta pengguna untuk memasukkan umur mereka.
8. `int umur = scanner.nextInt();`: Ini adalah pernyataan yang membaca sebuah bilangan bulat (integer) yang dimasukkan oleh pengguna dan menyimpannya dalam variabel `umur`.
9. `System.out.println("Halo, " + nama + "! Anda berusia " + umur + " tahun.");`: Ini adalah pernyataan yang mencetak pesan sambutan kepada pengguna, mencantumkan nama dan umur yang telah dimasukkan oleh pengguna.
10. `scanner.close();`: Ini adalah pernyataan yang menutup objek `Scanner` setelah selesai digunakan. Hal ini penting untuk menghindari kebocoran sumber daya.

Dengan kode ini, program akan menunggu masukan dari pengguna, kemudian membaca masukan tersebut, baik sebagai string (nama) atau bilangan bulat (umur), dan kemudian mencetak pesan sambutan dengan informasi yang dimasukkan oleh pengguna. Jadi, ini adalah contoh sederhana pengambilan dan pengolahan masukan pengguna dalam Java menggunakan kelas `Scanner`.

Anda juga dapat menggunakan kelas `BufferedReader` untuk membaca masukan dari keyboard. Berikut adalah contoh sederhana menggunakan `BufferedReader`:



```
8 *
9 * @author USER
10 * /
11 import java.io.BufferedReader;
12 import java.io.IOException;
13 import java.io.InputStreamReader;
14
15 public class InputExample {
16 public static void main(String[] args) throws IOException {
17 BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
18
19 System.out.print("Masukkan nama Anda: ");
20 String nama = reader.readLine(); // Membaca sebuah baris masukan sebagai string
21
22 System.out.print("Masukkan umur Anda: ");
23 int umur = Integer.parseInt(reader.readLine()); // Membaca sebuah baris masukan sebagai string dan mengonversinya ke int
24
25 System.out.println("Halo, " + nama + "! Anda berusia " + umur + " tahun.");
26
27 reader.close(); // Menutup objek BufferedReader
28 }
29 }
```

Gambar 21. Contoh lanjutan perintah masukan.

Dalam kedua contoh di atas, program akan menunggu masukan dari pengguna, kemudian membaca masukan tersebut dan menggunakannya dalam program. Input yang dibaca dapat digunakan untuk berbagai macam tugas dalam program, seperti perhitungan, pengambilan keputusan, dan sebagainya.

Penjelasan Source Coding:

1. `import java.io.BufferedReader;`: Ini adalah pernyataan impor yang mengimpor kelas `BufferedReader` dari paket `java.io`, yang digunakan untuk membaca masukan dari keyboard.
2. `import java.io.IOException;`: Ini adalah pernyataan impor yang mengimpor kelas `IOException` dari paket `java.io`, yang berkaitan dengan penanganan kesalahan masukan/keluaran.
3. `import java.io.InputStreamReader;`: Ini adalah pernyataan impor yang mengimpor kelas `InputStreamReader` dari paket `java.io`, yang digunakan untuk membungkus `System.in` sehingga dapat digunakan oleh `BufferedReader` untuk membaca masukan dari keyboard.
4. `public class InputExample {`: Ini adalah deklarasi kelas Java utama yang bernama `InputExample`.
5. `public static void main(String[] args) throws IOException {`: Ini adalah metode `main` yang merupakan titik masuk program. Metode ini mendeklarasikan bahwa ia dapat melemparkan pengecualian `IOException`, yang dapat terjadi saat membaca masukan.

6. `BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));`: Ini adalah pembuatan objek `BufferedReader` yang digunakan untuk membaca masukan dari keyboard. Objek ini dibuat dengan menggunakan `System.in` sebagai aliran masukan.
7. `System.out.print("Masukkan nama Anda: ");`: Ini adalah pernyataan cetak yang meminta pengguna untuk memasukkan nama mereka.
8. `String nama = reader.readLine();`: Ini adalah pernyataan yang membaca sebuah baris masukan sebagai string yang dimasukkan oleh pengguna dan menyimpannya dalam variabel `nama`.
9. `System.out.print("Masukkan umur Anda: ");`: Ini adalah pernyataan cetak yang meminta pengguna untuk memasukkan umur mereka.
10. `int umur = Integer.parseInt(reader.readLine());`: Ini adalah pernyataan yang membaca sebuah baris masukan sebagai string, kemudian mengonversinya menjadi bilangan bulat (integer) dan menyimpannya dalam variabel `umur`.
11. `System.out.println("Halo, " + nama + "! Anda berusia " + umur + " tahun.");`: Ini adalah pernyataan yang mencetak pesan sambutan kepada pengguna, mencantumkan nama dan umur yang telah dimasukkan.
12. `reader.close();`: Ini adalah pernyataan yang menutup objek `BufferedReader` setelah selesai digunakan. Hal ini penting untuk menghindari kebocoran sumber daya.

Dengan kode ini, program akan menunggu masukan dari pengguna, kemudian membaca masukan tersebut, mengonversi umur menjadi bilangan bulat, dan mencetak pesan sambutan dengan informasi yang dimasukkan oleh pengguna. Jadi, ini adalah contoh sederhana pengambilan dan pengolahan masukan pengguna dalam Java.

## 5.2. Perintah Seleksi Kondisi

Penyeleksian kondisi (conditional statements) dalam program Java adalah kemampuan untuk menjalankan kode tertentu berdasarkan evaluasi kondisi tertentu. Dengan kata lain, Anda dapat mengontrol alur program dengan melakukan pengujian kondisi tertentu dan menjalankan pernyataan berdasarkan hasil dari pengujian tersebut. Struktur kondisi digunakan untuk mengarahkan perjalanan suatu proses (Solichin 2016). Java menyediakan beberapa pernyataan kondisional yang umum digunakan, yaitu:

1. Pernyataan if: Pernyataan if digunakan untuk menjalankan blok kode jika kondisi yang diuji benar (true). Contoh:

```
java Copy code

int angka = 10;
if (angka > 5) {
 System.out.println("Angka lebih besar dari 5.");
}
```

2. Pernyataan if-else: Pernyataan if-else digunakan untuk menjalankan blok kode dalam if jika kondisi yang diuji benar (true) atau blok kode dalam else jika kondisi salah (false). Contoh:

```
java Copy code

int angka = 3;
if (angka > 5) {
 System.out.println("Angka lebih besar dari 5.");
} else {
 System.out.println("Angka tidak lebih besar dari 5.");
}
```

3. Pernyataan if-else if-else: Pernyataan ini digunakan untuk menguji beberapa kondisi berturut-turut dan menjalankan blok kode yang sesuai dengan kondisi yang pertama yang benar (true). Contoh:

```
java Copy code

int angka = 5;
if (angka > 10) {
 System.out.println("Angka lebih besar dari 10.");
} else if (angka > 5) {
 System.out.println("Angka lebih besar dari 5.");
} else {
 System.out.println("Angka kurang dari atau sama dengan 5.");
}
```

4. Pernyataan switch: Pernyataan switch digunakan untuk membuat seleksi berdasarkan nilai dari ekspresi tertentu. Contoh:

```

java Copy code

int hari = 2;
switch (hari) {
 case 1:
 System.out.println("Minggu");
 break;
 case 2:
 System.out.println("Senin");
 break;
 // ... kasus lainnya ...
 default:
 System.out.println("Hari tidak valid");
}

```

Pernyataan kondisional ini memungkinkan Anda untuk mengontrol jalannya program berdasarkan kondisi tertentu, sehingga Anda dapat melakukan perhitungan, pengambilan keputusan, dan alur program yang berbeda tergantung pada kondisi saat menjalankan program Java Anda.

### 5.3. Perintah Perulangan Pada Pemrograman Java

Perulangan dalam pemrograman Java adalah sebuah konsep yang memungkinkan Anda untuk menjalankan serangkaian instruksi atau blok kode secara berulang-ulang selama kondisi tertentu terpenuhi. Ini memungkinkan Anda untuk mengotomatisasi tugas-tugas yang memerlukan pengulangan, seperti mengulangi operasi pada setiap elemen dalam sebuah array atau menjalankan kode berulang kali sampai kondisi tertentu tercapai. Dalam Java, ada beberapa jenis perulangan yang dapat Anda gunakan:

1. **Perulangan `for`**: Digunakan ketika Anda tahu seberapa banyak kali kode harus diulang. Biasanya, Anda akan menggunakannya untuk mengiterasi melalui elemen-elemen dalam sebuah array atau untuk melakukan tugas-tugas yang berulang sejumlah tertentu. Contoh penggunaan perulangan `for`:

```

``java
for (int i = 0; i < 5; i++) {
 System.out.println("Iterasi ke-" + i);
}
...

```

2. **Perulangan `while`**: Digunakan ketika Anda tidak tahu berapa kali kode harus diulang, tetapi kode akan terus diulang selama kondisi tertentu benar. Contoh penggunaan perulangan `while`:

```
``java
int i = 0;
while (i < 5) {
 System.out.println("Iterasi ke-" + i);
 i++;
}
```

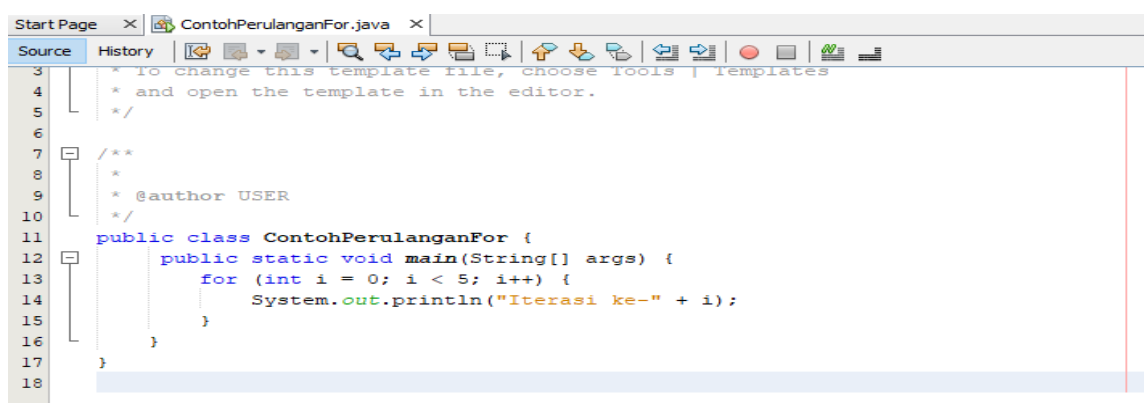
3. **Perulangan `do-while`**: Sama seperti `while`, namun memastikan bahwa setidaknya satu iterasi akan dijalankan bahkan jika kondisinya salah dari awal. Contoh penggunaan perulangan `do-while`:

```
``java
int i = 0;
do {
 System.out.println("Iterasi ke-" + i);
 i++;
} while (i < 5);
```

Perulangan memungkinkan Anda untuk mengotomatisasi tugas-tugas repetitif dalam pemrograman Java, dan pemilihan jenis perulangan yang tepat akan bergantung pada kebutuhan dan kondisi kode Anda.

Berikut contoh latihan perintah perulangan pada Java NetBeans:

### 1. Contoh Perulangan “For”



Gambar 22. Contoh Script Perulangan For

Berikut penjelasan dari script diatas:

- a. `public class ContohPerulanganFor {`: Ini adalah awal dari definisi kelas Java yang dinamai "ContohPerulanganFor". Semua kode Java harus berada dalam kelas, dan nama kelas harus sesuai dengan nama file Java yang disimpan dalam.

- b. `public static void main(String[] args) {`: Ini adalah metode utama dalam kelas. Setiap program Java harus memiliki metode `main` yang merupakan titik awal eksekusi program. Dalam kasus ini, metode `main` menerima argumen dalam bentuk array string (`String[] args`).
- c. `for (int i = 0; i < 5; i++) {`: Ini adalah deklarasi perulangan `for`. Di dalam tanda kurung, Anda memiliki tiga bagian:
  - o `int i = 0;`: Inisialisasi variabel `i` dengan nilai awal 0. Variabel `i` akan digunakan sebagai penghitung iterasi.
  - o `i < 5;`: Kondisi yang akan dievaluasi setiap kali perulangan dilakukan. Perulangan akan terus berlanjut selama kondisi ini benar (nilai `i` kurang dari 5).
  - o `i++`: Ekspresi yang dieksekusi setiap kali iterasi selesai. Ini meningkatkan nilai `i` sebanyak 1 setiap kali perulangan selesai.
- d. `{`: Kurung kurawal membuka untuk menandai awal dari blok kode yang akan diulang.
- e. `System.out.println("Iterasi ke-" + i);`: Ini adalah perintah yang akan dijalankan pada setiap iterasi perulangan. Ini mencetak teks "Iterasi ke-" diikuti oleh nilai `i` saat ini ke konsol.
- f. `}`: Kurung kurawal penutup menandai akhir dari blok kode perulangan.

Jadi, kode ini akan mencetak pesan "Iterasi ke-0", "Iterasi ke-1", "Iterasi ke-2", "Iterasi ke-3", dan "Iterasi ke-4" ke konsol, karena variabel `i` akan meningkat dari 0 hingga 4 dalam perulangan. Setelah `i` mencapai 5, kondisi `i < 5` menjadi salah, dan perulangan akan berhenti.

## 2. Contoh Perulangan “While”

Gambar 23. Contoh Perulangan While

Berikut penjelasan code script diatas:

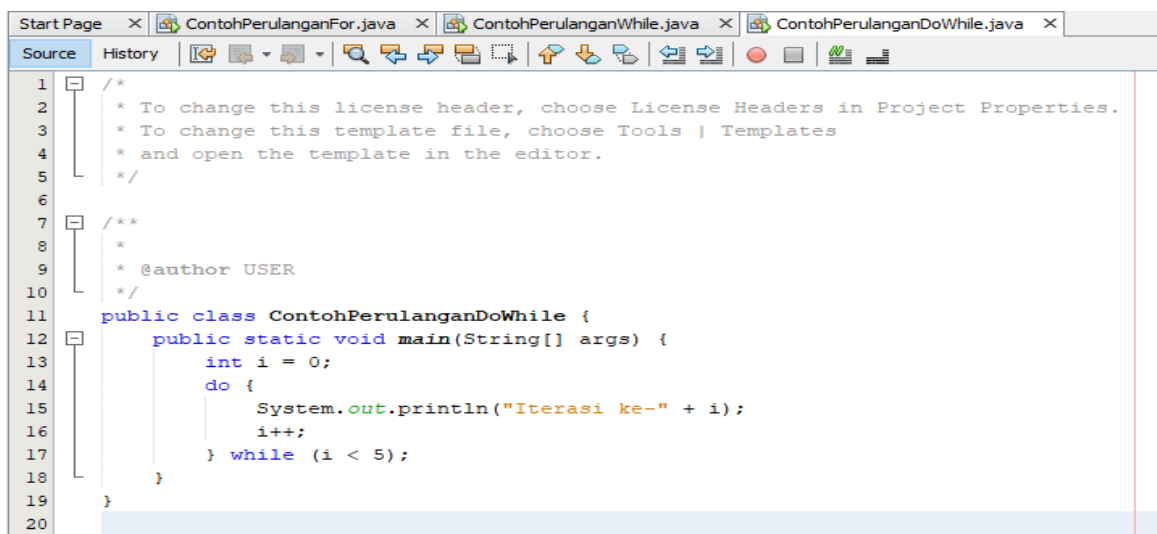
- a. `public class ContohPerulanganWhile {`: Ini adalah awal dari definisi kelas Java yang dinamai "ContohPerulanganWhile". Semua kode Java harus berada dalam kelas, dan nama kelas harus sesuai dengan nama file Java yang disimpan dalam.



- b. `public static void main(String[] args) {`: Ini adalah metode utama dalam kelas. Setiap program Java harus memiliki metode `main` yang merupakan titik awal eksekusi program. Dalam kasus ini, metode `main` menerima argumen dalam bentuk array string (`String[] args`).
- c. `int i = 0;`: Ini adalah inisialisasi variabel `i` dengan nilai awal 0. Variabel `i` akan digunakan sebagai penghitung iterasi.
- d. `while (i < 5) {`: Ini adalah deklarasi perulangan `while`. Di dalam tanda kurung, Anda memiliki kondisi `i < 5`. Perulangan ini akan terus berlanjut selama kondisi ini benar (nilai `i` kurang dari 5).
- e. `{`: Kurung kurawal membuka untuk menandai awal dari blok kode yang akan diulang.
- f. `System.out.println("Iterasi ke-" + i);`: Ini adalah perintah yang akan dijalankan pada setiap iterasi perulangan. Ini mencetak teks "Iterasi ke-" diikuti oleh nilai `i` saat ini ke konsol.
- g. `i++;`: Ini adalah perintah yang meningkatkan nilai `i` sebanyak 1 pada setiap iterasi. Ini diperlukan agar kondisi perulangan (`i < 5`) akhirnya menjadi salah dan perulangan berhenti.
- h. `}`: Kurung kurawal penutup menandai akhir dari blok kode perulangan.

Dengan demikian, kode ini akan mencetak pesan "Iterasi ke-0", "Iterasi ke-1", "Iterasi ke-2", "Iterasi ke-3", dan "Iterasi ke-4" ke konsol karena nilai `i` mulai dari 0 dan ditingkatkan setiap kali perulangan selesai. Setelah `i` mencapai 5, kondisi `i < 5` menjadi salah, dan perulangan berhenti.

### 3. Contoh Perulangan “do-while”.



```
1 /*
2 * To change this license header, choose License Headers in Project Properties.
3 * To change this template file, choose Tools | Templates
4 * and open the template in the editor.
5 */
6
7 /**
8 *
9 * @author USER
10 */
11 public class ContohPerulanganDoWhile {
12 public static void main(String[] args) {
13 int i = 0;
14 do {
15 System.out.println("Iterasi ke-" + i);
16 i++;
17 } while (i < 5);
18 }
19 }
20
```

Gambar 24. Contoh Cript Perulangan do-while

Berikut penjelsan dari source code diatas:

- a. ``public class ContohPerulanganDoWhile {``: Ini adalah awal dari definisi kelas Java yang dinamai "ContohPerulanganDoWhile". Semua kode Java harus berada dalam kelas, dan nama kelas harus sesuai dengan nama file Java yang disimpan dalam.
- b. ``public static void main(String[] args) {``: Ini adalah metode utama dalam kelas. Setiap program Java harus memiliki metode ``main`` yang merupakan titik awal eksekusi program. Dalam kasus ini, metode ``main`` menerima argumen dalam bentuk array string (``String[] args``).
- c. ``int i = 0;``: Ini adalah inisialisasi variabel ``i`` dengan nilai awal 0. Variabel ``i`` akan digunakan sebagai penghitung iterasi.
- d. ``do {``: Ini adalah awal dari blok perulangan ``do-while``. Perbedaan utama antara ``do-while`` dan ``while`` adalah bahwa blok kode di dalam ``do`` akan dijalankan sekali setidaknya sebelum kondisi dicek.
- e. ``System.out.println("Iterasi ke-" + i);``: Ini adalah perintah yang akan dijalankan pada setiap iterasi perulangan. Ini mencetak teks "Iterasi ke-" diikuti oleh nilai ``i`` saat ini ke konsol.
- f. ``i++;``: Ini adalah perintah yang meningkatkan nilai ``i`` sebanyak 1 pada setiap iterasi. Ini diperlukan agar kondisi perulangan (``i < 5``) akhirnya menjadi salah dan perulangan berhenti.
- g. ``} while (i < 5);``: Ini adalah bagian akhir dari perulangan ``do-while``. Pada titik ini, kondisi ``i < 5`` dicek. Jika kondisi masih benar, perulangan akan kembali ke awal blok ``do`` dan melanjutkan eksekusi. Jika kondisi tersebut salah, perulangan akan berhenti.

Kode tersebut akan mencetak pesan "Iterasi ke-0", "Iterasi ke-1", "Iterasi ke-2", "Iterasi ke-3", dan "Iterasi ke-4" ke konsol karena blok ``do`` dijalankan sekali, dan kemudian kondisi ``i < 5`` dicek. Setelah itu, perulangan akan terus berlanjut selama kondisi tetap benar, dan nilai ``i`` akan terus ditingkatkan. Setelah ``i`` mencapai 5, kondisi ``i < 5`` menjadi salah, dan perulangan berhenti.

#### 5.4. Perintah Array Pada Pemrograman Java

Dalam bahasa pemrograman Java, sebuah array adalah struktur data yang digunakan untuk menyimpan kumpulan nilai atau elemen sejenis dalam satu variabel. Setiap elemen dalam array dapat diakses berdasarkan indeksnya. Indeks dimulai dari 0 untuk elemen pertama, 1 untuk elemen kedua, dan seterusnya. Berikut beberapa karakteristik penting tentang array di Java:

- a. Tipe Data Sama: Semua elemen dalam array harus memiliki tipe data yang sama. Misalnya, Anda dapat memiliki array bilangan bulat, array string, atau array objek tertentu, tetapi semua elemennya harus memiliki tipe data yang sama.
- b. Ukuran Tetap: Setelah sebuah array dibuat, ukurannya tetap dan tidak dapat diubah. Ini berarti Anda harus menentukan ukuran array saat deklarasi, dan array tersebut akan memiliki jumlah elemen yang tetap.
- c. Indeks Berurutan: Elemen-elemen dalam array diindeks secara berurutan, dimulai dari 0. Ini berarti Anda dapat mengakses elemen tertentu dengan menggunakan indeksnya.
- d. Deklarasi dan Inisialisasi: Anda dapat mendeklarasikan dan menginisialisasi array dengan berbagai cara. Misalnya, Anda bisa langsung menyebutkan elemen-elemennya saat deklarasi atau mengisi array setelah deklarasi menggunakan loop atau nilai-nilai tertentu.

Contoh deklarasi dan penggunaan array dalam Java:

```
// Deklarasi dan inisialisasi array integer dengan ukuran 5
```

```
int[] angka = new int[5];
```

```
// Mengisi elemen-elemen array
```

```
angka[0] = 10;
```

```
angka[1] = 20;
```

```
angka[2] = 30;
```

```
angka[3] = 40;
```

```
angka[4] = 50;
```

```
// Mengakses elemen-elemen array
```

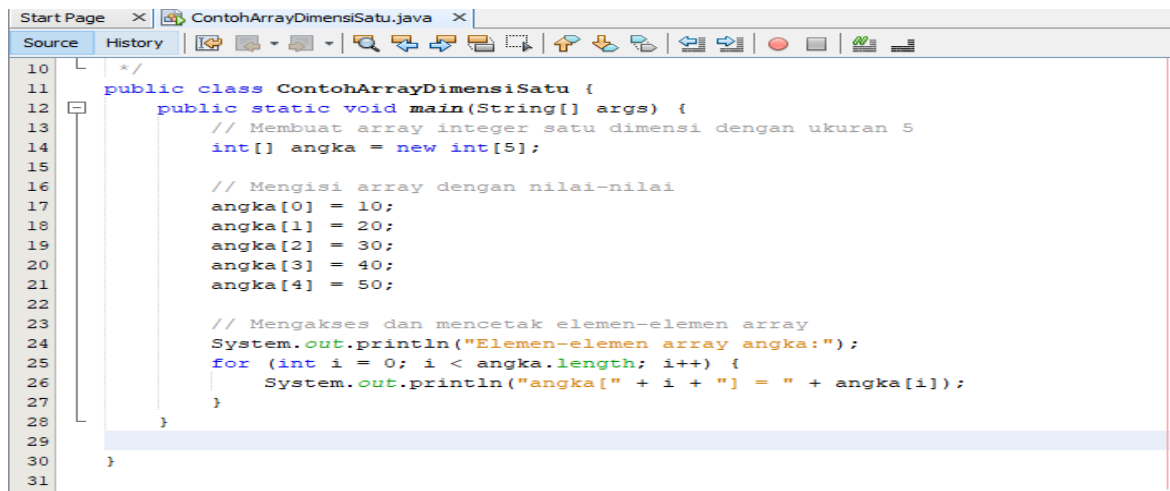
```
System.out.println(angka[0]); // Output: 10
```

```
System.out.println(angka[2]); // Output: 30
```

Java juga memiliki kelas utilitas seperti `ArrayList` yang memungkinkan Anda untuk membuat struktur data yang lebih fleksibel dengan ukuran yang dapat disesuaikan, tetapi array tetap menjadi struktur data dasar yang penting dalam bahasa ini.

## 1. Array Satu Dimensi

Array satu dimensi (1D array) adalah struktur data dalam pemrograman Java yang digunakan untuk menyimpan kumpulan elemen dengan tipe data yang sama dalam satu baris. Array satu dimensi memiliki satu indeks, sehingga Anda dapat mengakses elemen-elemen dalam array menggunakan satu angka indeks yang mengidentifikasi posisi elemen dalam array tersebut. Berikut beberapa contoh array satu dimensi dalam Java:

The image shows a screenshot of a Java IDE window titled 'ContohArrayDimensiSatu.java'. The code is as follows:

```
10 */
11 public class ContohArrayDimensiSatu {
12 public static void main(String[] args) {
13 // Membuat array integer satu dimensi dengan ukuran 5
14 int[] angka = new int[5];
15
16 // Mengisi array dengan nilai-nilai
17 angka[0] = 10;
18 angka[1] = 20;
19 angka[2] = 30;
20 angka[3] = 40;
21 angka[4] = 50;
22
23 // Mengakses dan mencetak elemen-elemen array
24 System.out.println("Elemen-elemen array angka:");
25 for (int i = 0; i < angka.length; i++) {
26 System.out.println("angka[" + i + "] = " + angka[i]);
27 }
28 }
29 }
30
31 }
```

Gambar 25. Contoh Source Coding Array Satu Dimensi.

Langkah-langkah dalam contoh di atas adalah sebagai berikut:

- Membuat array '**angka**' dengan ukuran 5 menggunakan deklarasi **int[] angka = new int[5];**.
- Mengisi array '**angka**' dengan nilai-nilai menggunakan pernyataan seperti **angka[0] = 10;**.
- Mengakses dan mencetak elemen-elemen array menggunakan loop '**for**' untuk mengunjungi setiap indeks array dan mencetak nilainya.

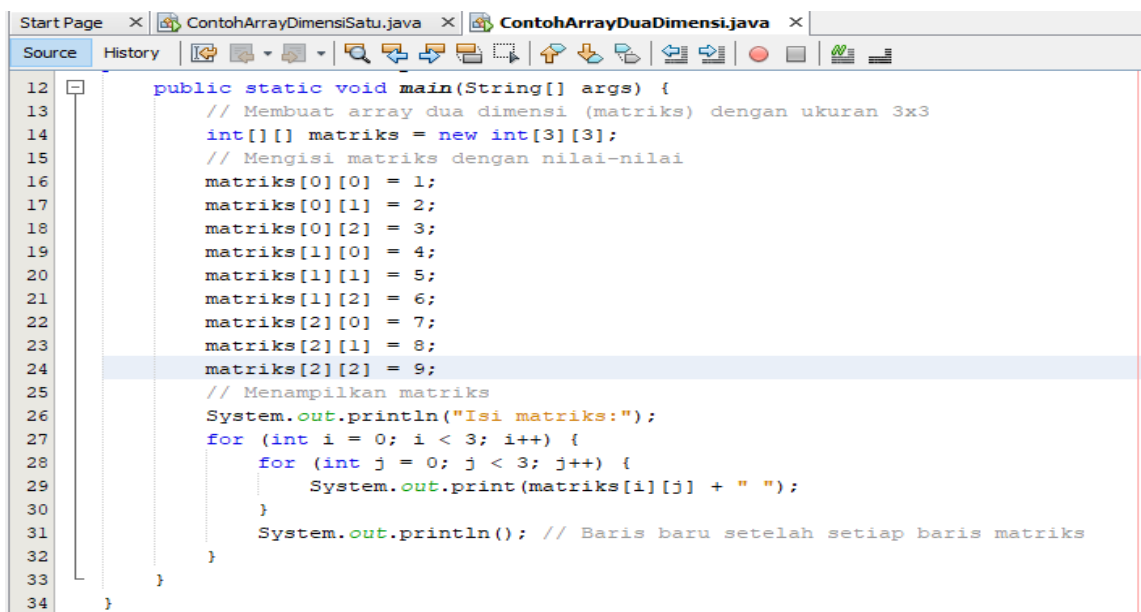
Penjelasan dari source coding diatas adalah:

- ``public class ContohArraySatuDimensi {``: Ini adalah awal dari definisi kelas Java yang dinamai "ContohArraySatuDimensi". Semua kode Java harus berada dalam kelas, dan nama kelas harus sesuai dengan nama file Java yang disimpan dalam.
- ``public static void main(String[] args) {``: Ini adalah metode utama dalam kelas. Setiap program Java harus memiliki metode ``main`` yang merupakan titik awal eksekusi program. Dalam kasus ini, metode ``main`` menerima argumen dalam bentuk array string (``String[] args``).
- ``int[] angka = new int[5];``: Ini adalah deklarasi dan inisialisasi array integer satu dimensi dengan nama ``angka`` dan ukuran 5. Ini berarti array ``angka`` akan memiliki lima elemen bertipe data integer.
- Mengisi array dengan nilai-nilai:
  - ``angka[0] = 10;``
  - ``angka[1] = 20;``
  - ``angka[2] = 30;``

- ``angka[3] = 40;``
  - ``angka[4] = 50;``
- e. ``System.out.println("Elemen-elemen array angka:");``: Ini adalah perintah yang mencetak pesan "Elemen-elemen array angka:" ke konsol sebagai label sebelum mencetak elemen-elemen array.
- f. Loop ``for`` digunakan untuk mengakses dan mencetak elemen-elemen array:
- ``for (int i = 0; i < angka.length; i++) {``: Ini adalah loop ``for`` yang akan mengulangi sebanyak elemen yang ada dalam array ``angka``.
  - ``System.out.println("angka[" + i + "] = " + angka[i]);``: Ini adalah perintah yang mencetak setiap elemen array dengan menggunakan indeks ``i``. Misalnya, untuk indeks 0, ini akan mencetak "angka[0] = 10" ke konsol.

## 2. Array Dua Dimensi

Array dua dimensi (2D array) adalah struktur data dalam pemrograman Java yang digunakan untuk menyimpan data dalam bentuk tabel dua dimensi, yang terdiri dari baris dan kolom. Dalam array dua dimensi, elemen-elemen disusun dalam grid atau matriks, di mana setiap elemen memiliki dua indeks: satu untuk baris dan satu untuk kolom. Ini memungkinkan Anda untuk mengakses data dalam grid berdasarkan koordinat baris dan kolomnya. Contoh umum penggunaan array dua dimensi adalah dalam merepresentasikan data seperti matriks, papan permainan, data tabel, atau gambar piksel (dalam konteks pengolahan citra). Dalam Java, Anda dapat mendeklarasikan dan menginisialisasi array dua dimensi sebagai berikut:



```

12 public static void main(String[] args) {
13 // Membuat array dua dimensi (matriks) dengan ukuran 3x3
14 int[][] matriks = new int[3][3];
15 // Mengisi matriks dengan nilai-nilai
16 matriks[0][0] = 1;
17 matriks[0][1] = 2;
18 matriks[0][2] = 3;
19 matriks[1][0] = 4;
20 matriks[1][1] = 5;
21 matriks[1][2] = 6;
22 matriks[2][0] = 7;
23 matriks[2][1] = 8;
24 matriks[2][2] = 9;
25 // Menampilkan matriks
26 System.out.println("Isi matriks:");
27 for (int i = 0; i < 3; i++) {
28 for (int j = 0; j < 3; j++) {
29 System.out.print(matriks[i][j] + " ");
30 }
31 System.out.println(); // Baris baru setelah setiap baris matriks
32 }
33 }
34

```

Gambar 26. Contoh Source Code Array Dua Dimensi

Pada source code diatas:

- a. Kita membuat sebuah matriks dua dimensi matriks dengan ukuran 3x3. Ini berarti matriks ini memiliki 3 baris dan 3 kolom.
- b. Kita mengisi matriks dengan nilai-nilai.
- c. Kemudian, kita menggunakan loop bersarang (nested loop) untuk mencetak isi matriks. Loop pertama mengontrol baris, dan loop kedua mengontrol kolom. Setelah mencetak semua elemen dalam satu baris, kita memasukkan baris baru menggunakan `System.out.println()`; sehingga matriks tercetak dengan format yang benar.

Penjelasan dari source code diatas sebagai berikut:

Kode yang Anda berikan adalah contoh penggunaan array dua dimensi (2D array) dalam bahasa pemrograman Java. Berikut adalah penjelasan baris per barisnya:

- a. `public class ContohArrayDuaDimensi {`: Ini adalah awal dari definisi kelas Java yang dinamai "ContohArrayDuaDimensi". Semua kode Java harus berada dalam kelas, dan nama kelas harus sesuai dengan nama file Java yang disimpan dalam.
- b. `public static void main(String[] args) {`: Ini adalah metode utama dalam kelas. Setiap program Java harus memiliki metode `main` yang merupakan titik awal eksekusi program. Dalam kasus ini, metode `main` menerima argumen dalam bentuk array string (`String[] args`).
- c. `int[][] matriks = new int[3][3];`: Ini adalah deklarasi dan inisialisasi array dua dimensi (matriks) dengan ukuran 3x3. Ini berarti matriks ini memiliki 3 baris dan 3 kolom, dan semua elemennya diinisialisasi dengan nilai 0 karena ini adalah nilai default untuk tipe data `int`.
- d. Mengisi matriks dengan nilai-nilai menggunakan indeks baris dan kolom, seperti `matriks[0][0] = 1;`, `matriks[0][1] = 2;`, dan seterusnya.
- e. `System.out.println("Isi matriks:");`: Ini adalah perintah yang mencetak pesan "Isi matriks:" ke konsol sebagai label sebelum mencetak isi matriks.
- f. Loop bersarang (nested loop) digunakan untuk mencetak isi matriks:
  - `for (int i = 0; i < 3; i++) {`: Ini adalah loop luar yang mengontrol baris matriks. Ini akan berjalan dari 0 hingga 2 (indeks baris).
  - `for (int j = 0; j < 3; j++) {`: Ini adalah loop dalam yang mengontrol kolom matriks. Ini juga akan berjalan dari 0 hingga 2 (indeks kolom).

- `System.out.print(matriks[i][j] + " ");`: Ini mencetak elemen matriks pada posisi (i, j) dengan spasi sebagai pemisah antara elemen-elemen dalam satu baris.
- g. `System.out.println();`: Ini mencetak baris baru setelah mencetak semua elemen dalam satu baris matriks.

Dalam contoh ini, matriks 3x3 diisi dengan nilai-nilai dan kemudian dicetak ke konsol dalam format yang sesuai dengan baris dan kolom.

## BAB VI

### CONTOH LATIHAN STUDI KASUS

Pada bab ini, akan dibahas studi kasus atau latihan-latihan mengenai penggunaan dan penerapan konsep OOP dan Java khususnya menggunakan NetBeans. Berikut beberapa studi kasus yang dapat diuji coba pada project masing-masing.

#### 1. Studi Kasus Soal 1

Berikut adalah sebuah studi kasus sederhana penggunaan dan penerapan konsep Pemrograman Berorientasi Objek (OOP) dalam Java menggunakan NetBeans. Dalam contoh ini, kita akan membuat kelas Mahasiswa dan MainApp untuk merepresentasikan data mahasiswa dan menjalankan aplikasi untuk menambahkan, menampilkan, dan mencari mahasiswa.

##### a. Class Mahasiswa

Kelas **Mahasiswa** akan digunakan untuk merepresentasikan mahasiswa dengan atribut seperti nama, NIM, dan nilai IPK. Berikut source code Class Mahasiswa:

```
public class Mahasiswa {
 private String nama;
 private String nim;
 private double ipk;

 public Mahasiswa(String nama, String nim, double ipk) {
 this.nama = nama;
 this.nim = nim;
 this.ipk = ipk;
 }

 public String getNama() {
 return nama;
 }

 public String getNim() {
 return nim;
 }

 public double getIpk() {
 return ipk;
 }

 @Override
 public String toString() {
 return "Mahasiswa{" + "nama=" + nama + ", nim=" + nim + ", ipk=" + ipk + '}';
 }
}
```



b. Class MainApp

Kelas **MainApp** akan berfungsi sebagai aplikasi utama untuk mengelola objek mahasiswa. Berikut source code pada class MainApp:

```
import java.util.ArrayList;
import java.util.List;

public class MainApp {
 public static void main(String[] args) {
 List<Mahasiswa> daftarMahasiswa = new ArrayList<>();

 // Menambahkan beberapa mahasiswa
 daftarMahasiswa.add(new Mahasiswa("John", "12345", 3.5));
 daftarMahasiswa.add(new Mahasiswa("Jane", "67890", 3.8));
 daftarMahasiswa.add(new Mahasiswa("Bob", "54321", 3.2));

 // Menampilkan semua mahasiswa
 System.out.println("Daftar Mahasiswa:");
 for (Mahasiswa mahasiswa : daftarMahasiswa) {
 System.out.println(mahasiswa);
 }

 // Mencari mahasiswa berdasarkan NIM
 String nimCari = "67890";
 for (Mahasiswa mahasiswa : daftarMahasiswa) {
 if (mahasiswa.getNim().equals(nimCari)) {
 System.out.println("\nMahasiswa dengan NIM " + nimCari + "
ditemukan:");
 System.out.println(mahasiswa);
 break;
 }
 }
 }
}
```

Dalam MainApp, kita membuat sebuah ArrayList daftarMahasiswa untuk menyimpan objek mahasiswa. Kemudian, kita menambahkan beberapa mahasiswa ke dalam list, menampilkan seluruh daftar mahasiswa, dan mencari mahasiswa berdasarkan NIM.

Jalankan program ini dalam NetBeans, dan Anda akan melihat bagaimana konsep OOP digunakan untuk mengorganisasi kode dan data dalam sebuah aplikasi. Dalam contoh ini, kita menggunakan kelas Mahasiswa sebagai abstraksi untuk objek mahasiswa dan MainApp sebagai pengelola aplikasi utama.

## 2. Studi Kasus Soal 2

Berikut adalah contoh sederhana studi kasus penjualan menggunakan konsep Pemrograman Berorientasi Objek (OOP) dalam Java dengan NetBeans. Dalam contoh ini, kita akan membuat beberapa kelas yang merepresentasikan produk, pelanggan, dan pesanan penjualan. Selain itu, kita juga akan membuat kelas utama (MainApp) untuk menjalankan aplikasi penjualan.

a. Class Produk

Kelas **Produk** akan merepresentasikan produk yang akan dijual, dengan atribut seperti ID produk, nama produk, harga, dan stok. Berikut Source Code pada Class Produk:

```
public class Produk {
 private int id;
 private String nama;
 private double harga;
 private int stok;

 public Produk(int id, String nama, double harga, int stok) {
 this.id = id;
 this.nama = nama;
 this.harga = harga;
 this.stok = stok;
 }

 public int getId() {
 return id;
 }

 public String getNama() {
 return nama;
 }

 public double getHarga() {
 return harga;
 }

 public int getStok() {
 return stok;
 }

 public void kurangiStok(int jumlah) {
 if (jumlah <= stok) {
 stok -= jumlah;
 } else {
 System.out.println("Stok tidak mencukupi.");
 }
 }

 @Override
```

```

 public String toString() {
 return "Produk{" + "id=" + id + ", nama=" + nama + ", harga=" + harga + ", stok="
+ stok + '}';
 }
}

```

b. Class Pelanggan

Kelas **Pelanggan** akan merepresentasikan pelanggan yang melakukan pembelian, dengan atribut seperti ID pelanggan, nama, dan alamat. Berikut source code class Pelanggan:

```

public class Pelanggan {
 private int id;
 private String nama;
 private String alamat;

 public Pelanggan(int id, String nama, String alamat) {
 this.id = id;
 this.nama = nama;
 this.alamat = alamat;
 }

 public int getId() {
 return id;
 }

 public String getNama() {
 return nama;
 }

 public String getAlamat() {
 return alamat;
 }

 @Override
 public String toString() {
 return "Pelanggan{" + "id=" + id + ", nama=" + nama + ", alamat=" + alamat +
 '}';
 }
}

```

c. Class Pesanan Penjualan

Kelas **PesananPenjualan** akan merepresentasikan pesanan penjualan yang berisi informasi produk yang dibeli, jumlahnya, pelanggan yang melakukan pembelian, dan total harga. Berikut Source Code Class PesananPenjualan:

```

import java.util.ArrayList;
import java.util.List;

```

```

public class PesananPenjualan {
 private int nomorPesanan;
 private Pelanggan pelanggan;
 private List<Produk> produkDibeli;
 private List<Integer> jumlahProduk;
 private double totalHarga;

 public PesananPenjualan(int nomorPesanan, Pelanggan pelanggan) {
 this.nomorPesanan = nomorPesanan;
 this.pelanggan = pelanggan;
 this.produkDibeli = new ArrayList<>();
 this.jumlahProduk = new ArrayList<>();
 this.totalHarga = 0.0;
 }

 public void tambahProduk(Produk produk, int jumlah) {
 if (produk.getStok() >= jumlah) {
 produkDibeli.add(produk);
 jumlahProduk.add(jumlah);
 totalHarga += produk.getHarga() * jumlah;
 produk.kurangiStok(jumlah);
 } else {
 System.out.println("Stok produk tidak mencukupi.");
 }
 }

 public double getTotalHarga() {
 return totalHarga;
 }

 @Override
 public String toString() {
 return "PesananPenjualan{" + "nomorPesanan=" + nomorPesanan + ",
 pelanggan=" + pelanggan +
 ", totalHarga=" + totalHarga + '}';
 }
}

```

#### d. Class MainApp

Kelas **MainApp** adalah kelas utama yang akan menjalankan aplikasi penjualan.

Berikut source code Class MainApp:

```

public class MainApp {
 public static void main(String[] args) {
 // Membuat beberapa produk
 Produk produk1 = new Produk(1, "Laptop", 1000.0, 10);
 Produk produk2 = new Produk(2, "Smartphone", 500.0, 20);

 // Membuat pelanggan
 }
}

```

```

Pelanggan pelanggan1 = new Pelanggan(101, "John", "Jalan ABC No. 123");

// Membuat pesanan penjualan
PesananPenjualan pesanan1 = new PesananPenjualan(1001, pelanggan1);
pesanan1.tambahProduk(produk1, 2);
pesanan1.tambahProduk(produk2, 3);

// Menampilkan informasi pesanan
System.out.println("Informasi Pesanan Penjualan:");
System.out.println(pesanan1);

// Menampilkan stok produk setelah pesanan
System.out.println("\nStok Produk setelah Pesanan:");
System.out.println(produk1);
System.out.println(produk2);
 }
}

```

### 3. Studi Kasus Soal 3

Berikut adalah contoh sederhana studi kasus penggajian menggunakan konsep Pemrograman Berorientasi Objek (OOP) dalam Java dengan NetBeans. Dalam contoh ini, kita akan membuat beberapa kelas yang merepresentasikan karyawan dan penggajian mereka. Kami juga akan menciptakan kelas utama (MainApp) untuk menjalankan aplikasi penggajian.

#### a. Class Karyawan

Kelas **Karyawan** akan merepresentasikan karyawan dengan atribut seperti ID karyawan, nama, jabatan, gaji pokok, dan jumlah jam kerja. Berikut Source Code class Karyawan:

```

public class Karyawan {
 private int id;
 private String nama;
 private String jabatan;
 private double gajiPokok;
 private int jamKerja;

 public Karyawan(int id, String nama, String jabatan, double gajiPokok, int jamKerja) {
 this.id = id;
 this.nama = nama;
 this.jabatan = jabatan;
 this.gajiPokok = gajiPokok;
 this.jamKerja = jamKerja;
 }

 public double hitungGaji() {
 // Menghitung gaji berdasarkan gaji pokok dan bonus (jika ada)
 double gaji = gajiPokok;
 if (jabatan.equalsIgnoreCase("manager")) {
 gaji += 0.2 * gajiPokok; // Bonus 20% untuk manajer
 }
 }
}

```

```

 }
 return gaji;
}

@Override
public String toString() {
 return "Karyawan{" + "id=" + id + ", nama=" + nama + ", jabatan=" + jabatan +
 ", gajiPokok=" + gajiPokok + ", jamKerja=" + jamKerja + '}';
}
}

```

b. Class MainApp

Kelas **MainApp** adalah kelas utama yang akan menjalankan aplikasi penggajian. Berikut source code pada Class MainApp:

```

import java.util.ArrayList;
import java.util.List;

public class MainApp {
 public static void main(String[] args) {
 // Membuat beberapa karyawan
 Karyawan karyawan1 = new Karyawan(101, "John Doe", "Staff", 3000.0, 40);
 Karyawan karyawan2 = new Karyawan(102, "Jane Smith", "Manager", 5000.0, 45);

 // Menampilkan informasi karyawan
 System.out.println("Informasi Karyawan:");
 System.out.println(karyawan1);
 System.out.println(karyawan2);

 // Menghitung gaji karyawan
 double gajiKaryawan1 = karyawan1.hitungGaji();
 double gajiKaryawan2 = karyawan2.hitungGaji();

 // Menampilkan gaji karyawan
 System.out.println("\nGaji Karyawan:");
 System.out.println("Gaji " + karyawan1.getNama() + ": $" + gajiKaryawan1);
 System.out.println("Gaji " + karyawan2.getNama() + ": $" + gajiKaryawan2);
 }
}

```

Dalam MainApp, kita membuat dua objek karyawan, menampilkan informasi karyawan, menghitung gaji mereka berdasarkan jabatan, dan menampilkan gaji karyawan. Jalankan program ini dalam NetBeans, dan Anda akan melihat bagaimana konsep OOP digunakan untuk mengorganisasi data dan logika penggajian. Dalam contoh ini, kita menggunakan kelas Karyawan sebagai abstraksi untuk objek karyawan dan MainApp sebagai pengelola aplikasi utama.

## DAFTAR PUSTAKA

- Enterprise, Jubilee. 2016. *Belajar Java, Database, Dan NetBeans Dari Nol*. Elex Media Komputindo.
- Hartono, Budi. 2023. *Pemrograman Berorientasi Objek Dengan Java BlueJ*. Yayasan Prima Agus Teknik Bekerjasama Dengan Universitas STEKOM.
- Solichin, Achmad. 2016. *Pemrograman Web Dengan PHP Dan MySQL*. Penerbit Budi Luhur.
- Yeyi Gusla Nengsih, S.Kom., M.Kom, Jamaludin, M.Kom., CBPA, Nani Krisnawaty Tachjar, S.Kom., M.T., Bella Hardiyana, S.Kom, M.Kom, Yulizar Widiatama, M.Eng, Mohammad Ridwan, S. Kom, M. Kom, Sitti Aisa, S.Kom., M.T, Nurul Aini, S.Kom., M.T, Sukisno, S. Kom, M. Kom. 2022. *Konsep Algoritma Dan Pemrograman : Mengenal Konsep Dasar Dan Praktis Dalam Bahasa Pascal Dan C*. Indie Press.